

I want to learn about AES so i can implement it in hardware as a peripheral for a risc-v core. How might i best start to learn?

Implementing AES (Advanced Encryption Standard) as a hardware peripheral is a fantastic way to bridge cryptography, digital logic design, and computer architecture. To build a functional accelerator, you need to understand the algorithm's specific transformations well enough to map them to logic gates, while also designing a control interface that your RISC-V core can talk to.

AES is a symmetric block cipher that processes data in fixed 128-bit blocks. It supports key sizes of 128, 192, or 256 bits. The encryption process consists of repeating a set of mathematical transformations—called "rounds"—multiple times.

To get you started on the right track, we can approach this from three different angles depending on your current background. Which of these sounds like the best starting point for you?

1. **The "Inner Workings" (The Algorithm):** We dive into the four core transformations of an AES round ([SubBytes](#), [ShiftRows](#), [MixColumns](#), [AddRoundKey](#)). This is essential for designing your datapath.
2. **Hardware Architecture Strategy:** We explore design trade-offs, such as choosing between an iterative loop (smaller area) versus a fully pipelined design (high throughput), and how to handle the S-Box (Look-up Table vs. Combinational Logic).
3. **The RISC-V Interface:** We look at how the processor will actually control your peripheral. This involves memory-mapped registers, status flags, and how to feed data in and out of the core.

Where would you like to begin?

Let's start with the "Inner Working". Also I want you to remember these starting points so we can visit the other points once I'm satisfied with the current point.

Ok, I'll remember that. You can always ask me to forget things or manage what I've saved for you on the Saved info page.

Let's dive into the **Inner Workings**.

To build this in hardware, the first mental shift we need to make is how we view the data. AES doesn't process the 128-bit block as a single long string of numbers. Instead, it organizes those 16 bytes into a 4×4 grid called the **State**.

Every round of encryption manipulates this grid.

Since you are designing this for a RISC-V core, you'll likely receive data from memory or a register (which is linear, 128 bits or 2×64 bits).

If our input 128-bit block is $b_0, b_1, b_2, \dots, b_{15}$, how do you think these bytes map into that 4×4 grid? Do they fill it **row-by-row** or **column-by-column**?

Row by row? Does it matter?

It is actually **column-by-column**.

And yes, it matters **critically**.

If you map the bytes row-by-row, your hardware will still "work" (signals will toggle), but it will produce the wrong ciphertext. This is because the subsequent operations rely on specific geometric relationships between the bytes.

Here is how the 128-bit input string (b_0 to b_{15}) maps to the State matrix:

$$\begin{bmatrix} b_0 & b_4 & b_8 & b_{12} \\ b_1 & b_5 & b_9 & b_{13} \\ b_2 & b_6 & b_{10} & b_{14} \\ b_3 & b_7 & b_{11} & b_{15} \end{bmatrix}$$

Why this matters for Hardware Design

In your Verilog/SystemVerilog code, this transformation is "free." It costs zero logic gates and zero area because it is just **wire routing**. You are simply assigning which bits of your 128-bit input bus connect to which byte of your internal state logic.

However, if you get this indexing wrong, the next steps will fail:

1. **ShiftRows** rotates the data **horizontally**.
2. **MixColumns** mixes the data **vertically**.

If your data is transposed (filled row-by-row), **ShiftRows** would accidentally mix data that belongs in different columns, and **MixColumns** would mix data that belongs in different rows.

The Round Logic

Now that we have our State matrix, the "Round" logic is just a combinational block that transforms this matrix.

An AES Round consists of four distinct steps, usually performed in this order:

1. **SubBytes** (Non-linear substitution)
2. **ShiftRows** (Permutation)
3. **MixColumns** (Linear mixing)
4. **AddRoundKey** (XOR with the key)

Since you are looking to implement this as a peripheral, **SubBytes** is usually the most critical part to decide on first because it dominates the area cost.

Shall we look at how to implement **SubBytes** efficiently in hardware?

I've heard there can be more than 128 bits as input. is that a different standard?

That is a very common point of confusion. The short answer is **no**—if it is "Standard AES," the input block is **strictly 128 bits**.

If you see "AES-192" or "AES-256," that number refers to the **Key Size**, not the input Data Size.

- **AES-128**: 128-bit Data, 128-bit Key
- **AES-256**: 128-bit Data, 256-bit Key

The "Secret Menu" (Rijndael)

AES was chosen from a cipher family called **Rijndael**. The original Rijndael algorithm *did* support larger block sizes (160, 192, 224, and 256 bits). However, when the US Government (NIST) standardized it as AES, they dropped all the larger block sizes.

If you implement a 256-bit data block, you are implementing "Rijndael-256," not "AES." It is a valid cipher, but it won't be compatible with standard software/libraries.

What if I have more than 128 bits of data?

In the real world, we obviously encrypt files larger than 128 bits (16 bytes). To do this, we don't change the hardware; we use a **Mode of Operation** (like CBC, CTR, or GCM).

- The hardware encrypts the first 128 bits.
- The hardware takes the result, mixes it with the next 128 bits, and encrypts again.
- Rinse and repeat.

Back to the Hardware

Since you are building a standard AES peripheral, you should fix your input data bus width to **128 bits** (or 2 cycles of 64 bits if your RISC-V bus is 64-bit).

Ready to look at **SubBytes**? This is where the famous "S-Box" comes in.

Not yet. One more question. If I were to implement an AES accelerator, would I need to implement those modes of operation?

The short answer is **no**, you do not need to implement them in hardware, especially for a first project.

You can design your hardware to be a "**Block Cipher Primitive**."

In this design strategy, your hardware does **one specific thing**: it takes 128 bits of data + a key, crunches the numbers, and spits out 128 bits of result. That is technically "ECB Mode" (Electronic Codebook), but it serves as the engine for all other modes.

The "Software-Assisted" Approach

If you build just the primitive, your RISC-V processor handles the "Mode of Operation" logic in software.

For example, if you want to perform **CBC (Cipher Block Chaining)**, your RISC-V code would look like this:

1. **Software:** Reads the next 128 bits of plaintext from RAM.
2. **Software:** XORs that plaintext with the "Previous Ciphertext" (or IV).
3. **Software:** Writes that result to your **AES Hardware**.
4. **Hardware:** Encrypts the block (performs the heavy lifting).
5. **Software:** Reads the result back.

This is a very common design pattern. It keeps your hardware simple (saving silicon area) while still offloading the most computationally expensive part (the AES rounds) to your accelerator.

When *would* you implement Modes in Hardware?

You would only implement modes (like CTR or GCM) in hardware if you need **streaming performance** (like a network card or SSD controller).

In those cases, you don't want the CPU waking up every 128 bits to do an XOR. You want to say, "Here is a 1GB buffer, encrypt the whole thing," and walk away. That requires the hardware to manage the counters and XORs itself.

My Recommendation

Stick to the **Block Primitive** (Raw AES) first.

It is much easier to debug. If you feed it **0x00...00** and get the wrong answer, you know exactly where the bug is. If you wrap it in complex mode logic immediately, debugging

becomes a nightmare.

Ready to dive into the **SubBytes** and the S-Box now?

yea

SubBytes: The "Scrambler"

This is the most famous part of AES. **SubBytes** is a non-linear byte substitution step that operates independently on each of the 16 bytes of the State.

Think of it as looking up a value in a dictionary. You take a byte (say, **0x53**), look it up in a fixed table (the **S-Box**), and replace it with the value found there (e.g., **0xED**).

Why is this step critical?

The "Confusion" Property: Without **SubBytes**, AES would be entirely linear (just XORs and shifts). If AES were linear, a hacker could solve for your Key using simple high-school algebra (systems of linear equations). The S-Box breaks that linearity, making the math incredibly hard to reverse without the Key.

Hardware Implementation: The 256-Byte Table

Since the S-Box is constant (it never changes), you don't calculate the math every time (unless you are extremely constrained on space). Instead, you hardcode the answers.

In hardware, you have two main ways to implement this:

1. The Look-Up Table (LUT) / ROM Approach (Recommended)

You treat the S-Box as a small Read-Only Memory.

- **Input:** 8-bit address (the byte you want to change).
- **Output:** 8-bit data (the substituted value).

In Verilog/SystemVerilog, this often looks like a giant **case** statement:

```
Verilog
function automatic [7:0] sbox(input [7:0] in_byte);
  case (in_byte)
    8'h00: sbox = 8'h63;
    8'h01: sbox = 8'h7c;
    // ... all 256 values ...
    8'hff: sbox = 8'h16;
  endcase
endfunction
```

2. The Combinational Logic Approach

You can actually calculate the value using boolean logic gates (based on Galois Field inversion

over $\$GF(2^8)$). This uses zero memory bits but lots of XOR/AND gates.

- **Pros:** useful for ASICs where ROM macros are annoying to place.
- **Cons:** Much harder to design and verify. **Skip this for now.**

The "Area vs. Speed" Decision

This is your first major architectural choice. You have 16 bytes to substitute. How many S-Box copies do you put in your hardware?

- **Option A: 16 Copies (Parallel)**
 - You instantiate 16 S-Box tables.
 - You can transform the entire 128-bit state in **1 clock cycle**.
 - *Cost:* Uses more LUTs/Logic.
- **Option B: 1 Copy (Iterative/Shared)**
 - You have only 1 S-Box. You feed it Byte 0, store the result, then feed it Byte 1, etc.
 - Takes **16 clock cycles** just to do this one step.
 - *Cost:* Tiny area, but very slow.

Recommendation: Since you are likely targeting an FPGA or a modern simulation, **go with Option A (16 Copies)**. S-Boxes are small enough that 16 of them won't break the bank, and the logic is much simpler because you don't need a counter to loop through them.

Once the bytes are substituted, we have to move them around. Ready to see **ShiftRows**?

In AES in the initial **AddRoundKey** stage is that round key calculated or just the input key

In the initial **AddRoundKey** stage of AES, the key used is just the input key (specifically, the first 128 bits of it). It is not a calculated/derived key.

Technically, this is defined as "Round Key 0" in the AES Key Schedule, but the calculation for the schedule begins by simply copying the original Cipher Key into the first set of words.

Here is the breakdown by AES version:

1. AES-128 (128-bit key)

- Input Key Length: 128 bits
- Round Key Length: 128 bits
- Key Used: The initial **AddRoundKey** uses the exact, entire input key. No modification has occurred yet.

2. AES-192 and AES-256

- Input Key Length: 192 bits or 256 bits
- Round Key Length: 128 bits (AES block size is always 128 bits)
- Key Used: The initial **AddRoundKey** uses the first 128 bits of your input key. The remaining bits of your input key are used to calculate subsequent round keys.

Why is it done this way?

The AES Key Expansion algorithm works by generating a long list of 32-bit words (denoted as $w_0, w_1, w_2, \dots$).

- The first N words are simply copied from your original key.
- Only after those first words are filled does the algorithm start using XOR, S-Boxes, and Round Constants (Rcon) to calculate new, "scrambled" keys for later rounds.

Summary: The very first operation in AES is essentially XORing your plaintext directly with your password (or the first 128 bits of it).

in aes128 the first addround key before we go into rounds is adding the input key and not a calculated key?

Yes, that is exactly correct.

In AES-128, the initial **AddRoundKey** step (often referred to as "Round 0") uses your original, raw input key directly. It does not use a calculated or derived key yet.

Here is why:

- **Key Schedule Generation:** The AES Key Expansion algorithm generates a total of 11 round keys (for rounds 0 through 10) from your original 128-bit key.
- **The First Step:** The algorithm's very first rule is to copy the original Cipher Key into the first slot of the Key Schedule (Round Key 0).
- **Subsequent Rounds:** It is only for Round Key 1 through Round Key 10 that the algorithm applies transformations (like **RotWord**, **SubWord**, and **Rcon**) to calculate new, derived keys.

So, before any mixing (SubBytes, ShiftRows, MixColumns) happens, the plaintext is simply XORed with your password.

Quick Comparison

Round	Key Used	Source
Initial Round (Round 0)	Input Key	Direct copy of the Cipher Key
Rounds 1–10	Calculated Keys	Derived using previous keys + S-Box + Constants

Would you like me to explain how the keys for Round 1 are calculated from that input key?

Help me learn AES because I want to create an AES accelerator peripheral to a risc-v processor.

Building a hardware accelerator is an ambitious project that bridges cryptography, digital logic, and computer architecture. To integrate an AES peripheral with a RISC-V core, you need to understand the algorithm not just as software code, but as a sequence of logical operations that can be mapped to gates and registers.

AES (Advanced Encryption Standard) is a symmetric block cipher that processes data in fixed 128-bit blocks. It organizes this data into a 4×4 grid of bytes called the State.

The encryption process involves repeating a set of four transformations (a "Round") multiple times. For a hardware accelerator, your primary design decisions will revolve around how to implement these transformations efficiently in silicon (or FPGA) and how to manage the data flow between the processor and your peripheral.

Here are three ways we can start dissecting AES for your hardware implementation. Which one would you like to tackle first?

1. **The Round Transformations (The Datapath):** We can break down the four core operations—*SubBytes*, *ShiftRows*, *MixColumns*, and *AddRoundKey*—to understand exactly what logic gates are required for each.
2. **Galois Field Arithmetic (The Logic Optimization):** AES doesn't use standard integer math; it uses finite field arithmetic ($GF(2^8)$). Understanding this is critical if you want to optimize your hardware area (e.g., calculating S-boxes on the fly vs. storing them in large lookup tables).
3. **Key Expansion (The Setup):** Before encryption begins, the input key must be expanded into a "Key Schedule." We can look at how this algorithm works and discuss whether your accelerator should handle this or offload it to the RISC-V software.

can we start a simpler level?

No problem! Let's zoom out and look at the "Big Picture" before diving into the math or logic gates.

Since you are building a hardware accelerator, it helps to think of AES not as a math formula, but as a scrambling machine.

Imagine AES is a mechanical box on an assembly line:

1. **Input:** You feed it a fixed-size chunk of data (plain text) and a secret Key.
2. **Process:** Inside the box, the data goes through a series of "scrambling layers." It doesn't just scramble once; it repeats the scrambling process 10 times (for a standard 128-bit key).
3. **Output:** The result is a chunk of encrypted data (ciphertext) that looks like random noise.

Your RISC-V processor is currently doing this "scrambling" by calculating one step at a time, which is slow. Your accelerator will be a dedicated circuit that does the heavy lifting much faster.

Which of these high-level concepts would you like to visualize first?

1. **The Data Block:** AES doesn't see data as a long string of numbers (like `0x1A4F...`), but rather as a 4×4 grid. We can look at how we shape the wires to fit this grid.
2. **The "Round" Loop:** We can look at the flow of the 10 rounds to understand the timing and control logic your accelerator will need.

the data block

Great choice. Before we can scramble the data, we have to know exactly what shape it takes inside our machine.

In the world of AES, we don't look at the 128-bit input as a long linear string of ones and zeros. Instead, we arrange those 16 bytes (since $128 \text{ bits} / 8 \text{ bits per byte} = 16 \text{ bytes}$) into a 4×4 grid called the State.

The Order Matters

When your RISC-V processor sends a 128-bit chunk of data to your accelerator, the bytes fill this grid in a specific pattern: column by column.

If your input bytes are $b_0, b_1, b_2, \dots, b_{15}$, they map into the grid like this:

Col 0	Col 1	Col 2	Col 3
b_0	b_4	b_8	b_{12}
b_1	b_5	b_9	b_{13}
b_2	b_6	b_{10}	b_{14}
b_3	b_7	b_{11}	b_{15}

Why this matters for your hardware

In your Verilog or VHDL design, you won't necessarily be building a 2D physical grid, but your logic will treat the data this way.

- Row Operations: Some steps will shift bytes left or right within a row.
- Column Operations: Other steps will mix bytes up and down within a column.

This grid structure allows AES to confuse the data in two dimensions (horizontal and vertical) very quickly.

To make sure we've got the mapping right before we start moving things around:

If the very first byte your processor sends is **0x55** (byte b_0) and the second byte is

0xFF (byte **\$b_1\$**), are these two bytes sitting next to each other in the same row, or stacked in the same column?

stacked in the same column

Exactly. They are stacked vertically in Column 0.

This "column-major" order is crucial for your hardware interface. If your RISC-V processor writes a 32-bit word to your accelerator, that word effectively becomes one full column of the State.

The Scrambling Loop (The "Rounds")

Now that we have our data sitting in the grid, we start the scrambling machinery. We don't just run the operations once; we loop through them repeatedly to ensure the data is thoroughly obfuscated. These loops are called Rounds.

For the standard AES-128 (which uses a 128-bit key) that you are likely targeting, do you happen to know how many times we have to repeat this loop?

10?

Spot on! 🎯

For **AES-128**, we run the loop exactly **10 times**.

There is one critical "gotcha" for your hardware control logic, though: **Round 10 is different.**

*** Rounds 1–9:** Perform all 4 internal steps.

*** Round 10:** Skips one specific step (MixColumns).

So, your hardware state machine will need a counter (0 to 9) and a special "done" state that handles that slightly different final round.

Inside the Round

Now we get to the meat of the accelerator. Inside every standard round, the state goes through these four transformations in order:

1. **SubBytes** (Substitute Bytes)
2. **ShiftRows**
3. **MixColumns**
4. **AddRoundKey**

Let's start with the first one: ****SubBytes****.

This step uses something called an ****S-Box**** (Substitution Box), which is basically a fixed table of values. We take a byte from our state, look it up in the table, and replace it with the new value.

From a hardware perspective, if we are replacing every single byte in our grid using this same table, do we need to process them one by one, or can we process all 16 bytes at the exact same time?

we can process them in parallel?

Yes! You have hit on the exact reason why we build hardware accelerators. 🚀

In your RISC-V processor (software), the CPU has to look up one byte, replace it, move to the next address, look it up, replace it, and so on. It takes at least 16 separate instructions (plus overhead) to finish this step.

In your hardware accelerator, you can simply instantiate 16 copies of the S-Box logic.

- **Software: 1 byte at a time (Sequential).**
- **Hardware: 16 bytes at the exact same instant (Parallel).**

This massive parallelism is where you get your speed boost. The trade-off is Area (silicon real estate). You have to pay the cost of building 16 S-Boxes on your chip instead of just one.

Step 2: ShiftRows

Now that we have substituted all the bytes in parallel, our data is "confused" (the values are changed), but the bytes are still in their original positions. We need to move them around to "diffuse" the data.

The next step is ShiftRows.

Based on the name, and remembering that our data is in a 4×4 grid, what do you think happens to the first row vs. the second row in this step?

We shift them down? Also I have a question: So if byte 0 is 0xFF, it looks up 0xFF in a table? where does that come from?

Let's tackle your guess about the shifting first, then dive into that S-Box mystery.

1. ShiftRows: Left, not Down

You guessed "shift them down," which is a very logical guess since we just talked about columns! However, ShiftRows actually works horizontally (left).

Think of the rows as sliding cyclic belts.

- Row 0: Does not move.
- Row 1: Shifts left by 1 byte.
- Row 2: Shifts left by 2 bytes.
- Row 3: Shifts left by 3 bytes.

Why this is great for your accelerator:

In hardware, "shifting" by a fixed amount doesn't actually require logic gates or clock cycles. It just requires wires. You physically route the wire from the output of "Row 1, Byte 0" to the input of "Row 1, Byte 3". It costs almost zero silicon area!

2. The S-Box: Where do the values come from?

To answer your second question: Yes, if the input is 0xFF, the hardware uses 0xFF (which is 255) as the address to look up the output value.

As for "Where does that value come from?":

It isn't a random list of numbers. The S-Box is derived from a specific mathematical recipe using Galois Field Arithmetic ($\text{GF}(2^8)$).

1. Inversion: We take the input byte and find its "multiplicative inverse" in this finite field (like finding $1/x$, but with special polynomial math).
2. Affine Transformation: We scramble the bits of that result using a specific matrix operation.

This specific math is chosen because it is non-linear, meaning it breaks any simple relationship between the input and output, making the encryption incredibly hard to crack.

Your Hardware Decision

Since you are building a peripheral, you have a major design choice here:

1. The "Table" Method (ROM/LUT): You pre-calculate the values (or copy them from a standard list) and store them in a read-only memory block on your chip. This is

faster to design but takes up more space.

2. The "Math" Method (Combinational Logic): You build logic gates that actually calculate the math (inverse + affine) on the fly for every byte. This saves memory space but is much more complex to design.

For your first AES accelerator, which approach sounds more appealing to you?

Precalculating and storing them

That is a very practical choice. Using a pre-calculated Lookup Table (LUT) is the standard way to implement AES in hardware because it's fast and much harder to mess up than implementing complex polynomial math from scratch.

The Hardware: A ROM Table

In your accelerator, this will essentially look like a small block of Read-Only Memory (ROM). We call this the S-Box Table.

You will have 16 copies of this table (or a multi-port memory) so you can look up all 16 bytes of the State at once.

Addressing the Table

Now, we need to figure out the size of this memory block.

Since the input to the SubBytes step is a single byte (8 bits), and that byte acts as the "address" for our lookup, how many total entries (rows) must our table contain to handle every possible input byte?

256 entries or 16 rows of 16?

Spot on! 🎯

Since the input is 8 bits, there are $2^8 = 256$ possible patterns, so you need 256 entries.

Your intuition about "16 rows of 16" is actually how the S-Box is usually drawn in textbooks! They split the byte into two halves (nibbles):

- Top 4 bits: Select the Row (0–15).
- Bottom 4 bits: Select the Column (0–15).

Total Memory Size

Now, since every entry in this table provides an 8-bit replacement value, how many bytes of storage does a single S-Box table require?

256*8 bits? Also when u say bot and top bits, for a value of like 0x0f, what is top and bot bits?

Spot on again.

$256 \text{ entries} \times 8 \text{ bits per entry} = 2,048 \text{ bits}$.

In standard computer terms, that is just 256 Bytes (or 0.25 KB).

For a hardware chip, that is tiny! Even the smallest FPGA or ASIC has plenty of room for this.

Unpacking 0x0F

To find the "top" and "bottom" bits of 0x0F, we look at its binary representation. A hex digit represents 4 bits (a nibble).

0x0F in binary is 0000 1111.

Top 4 Bits (Row)	Bottom 4 Bits (Column)
0000	1111
Value: 0	Value: 15 (or F)

So, if your input byte is 0x0F, the hardware goes to Row 0, Column 15 of the S-Box table to find the replacement value.

The Hardware Challenge

Now we have a small table (256 bytes) and we know how to look things up. But remember our goal: Parallelism.

We want to substitute all 16 bytes of our State grid at the *exact same time* in one clock cycle.

If we put this table into a single block of standard RAM (which usually allows reading only 1 or 2 addresses at a time), can we successfully read 16 different values simultaneously?

No? because we need multiple ports(16) to read 16 different values

Exactly. Standard RAM blocks typically only allow one or two accesses at a time. Trying to build a "16-port" memory would be incredibly complex and slow.

However, we have a workaround because our table is so small. Instead of sharing one table, we can simply give every byte its own private table. We duplicate the S-Box 16 times.

Let's do a quick "cost" analysis to see if this is feasible for your chip.

If we have 16 copies of the table, and each copy is 256 bytes, what is the total memory size required in Kilobytes (KB)?

4kB?

You got it!

$256 \text{ bytes} \times 16 = 4096 \text{ bytes} = 4 \text{ KB}$.

In the world of hardware (even for small embedded chips), 4 KB of memory is a very reasonable "price" to pay for the massive speed increase of processing all 16 bytes at once.

Status Check

So far, we have designed the hardware for the first two steps of the Round:

1. SubBytes: 16 parallel S-Box tables (4 KB total).
2. ShiftRows: Just routing wires (effectively free!).

Step 3: MixColumns

Now things get a bit more mathematical. While *ShiftRows* moved bytes horizontally, *MixColumns* works vertically. It takes each column of 4 bytes and "mixes" them so that a small change in one byte affects all four bytes in that column.

This step treats each column as a vector of 4 numbers and multiplies it by a fixed 4×4 matrix.

$$\begin{bmatrix} d_0 \\ d_1 \\ d_2 \\ d_3 \end{bmatrix} = \begin{bmatrix} 2 & 3 & 1 & 1 \\ 1 & 2 & 3 & 1 \\ 1 & 1 & 2 & 3 \\ 3 & 1 & 1 & 2 \end{bmatrix} \begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

If you look at that matrix, you see the numbers 1, 2, and 3.

In normal integer math, multiplying by 1 is easy (it stays the same) and multiplying by 2 is just $x * 2$. But remember, AES uses that special Galois Field math.

In binary hardware, multiplying by 2 isn't standard multiplication. Do you happen to know the hardware trick for "multiplying by 2" in binary? (Hint: It involves shifting).

<<? but what about 3?

You are absolutely right: multiplying by 2 is just a left shift (<< 1).

However, there is that one pesky "Galois Field" rule: our numbers must fit in 8 bits. If you shift left and a '1' falls off the edge (overflow), you have to fix it by XORing the result with a magic number (0x1B).

So, what about multiplying by 3?

You don't need a "multiply by 3" circuit. You can build it out of what you already have!

In math, $3 = 2 + 1$.

Therefore:

$$\text{Input} \times 3 = (\text{Input} \times 2) + (\text{Input} \times 1)$$

In our hardware world, "Plus" means XOR. So the logic is simply:

$$\text{Input} \times 3 = (\text{Input} \ll 1 \text{ } \oplus \text{ } \text{Input})$$

The Hardware implications

This is great news for your silicon area. You don't need complex multipliers.

1. Block A: Build a small circuit that does the **x2** operation (Shift + conditional XOR).
2. Block B: To get **x3**, just take the output of Block A and XOR it with the original input.

Putting it together (MixColumns)

So, looking back at that matrix row:

$$d_0 = (b_0 \times 2) \oplus (b_1 \times 3) \oplus (b_2 \times 1) \oplus (b_3 \times 1)$$

Your hardware for calculating d_0 will look like a tree of XOR gates:

1. Take b_0 , run it through the x2 block.
2. Take b_1 , run it through the x2 block, then XOR with original b_1 .
3. Take b_2 (do nothing).
4. Take b_3 (do nothing).
5. XOR all four results together.

You repeat this logic 4 times (once for each row in the column) to finish the column.

Step 4: AddRoundKey (The easy part)

We are at the final step of the Round!

- 1. SubBytes (Done - Tables)**
- 2. ShiftRows (Done - Wires)**
- 3. MixColumns (Done - XOR tree)**
- 4. AddRoundKey**

This step takes your 128-bit State and combines it with a 128-bit "Round Key" (which is derived from the main password/key).

"Add" in Galois Field means XOR.

So, to combine your 128-bit State with the 128-bit Key, what specific hardware gate do we need, and how many of them?

128 xors?

Bingo. You need 128 XOR gates operating in parallel.

In terms of hardware cost, this is dirt cheap. XOR gates are tiny, and because there is no "carry" bit (like in normal addition), the signal zips through instantly.

The Full "Round" Engine

Congratulations! You have just mentally designed the datapath for the core AES engine.

Here is what your accelerator looks like so far:

1. Input: 128-bit State.
2. SubBytes: The signal splits into 16 paths, goes through 16 S-Box tables (4 KB ROM).
3. ShiftRows: The wires cross over each other to swap positions.
4. MixColumns: The wires go through a layer of "x2" and XOR gates to mix the columns.
5. AddRoundKey: The signal passes through a wall of 128 XOR gates.
6. Output: The result loops back to the beginning to start the next Round.

The Missing Piece: Key Expansion

We have one major problem left to solve.

We loop this engine 10 times. In the **AddRoundKey** step, we can't use the same key every time (that would be insecure). We need a unique 128-bit key for every single round.

Since we have the initial setup + 10 rounds, we need 11 unique keys in total.

This process of turning your one password (the "Master Key") into 11 separate keys is called Key Expansion.

Since you are attaching this to a RISC-V processor, you have a classic "Hardware vs. Software" trade-off to make:

Option A: The "Dumb" Peripheral (Software Assisted)

- How it works: Your RISC-V processor calculates all 11 keys in software *before* the encryption starts. It writes them into a bank of registers on your accelerator.
- Pros: Your hardware is much simpler. You don't need to build the "Key Expansion" logic in silicon.
- Cons: It takes time for the CPU to compute and transfer all that data (1408 bits

total) before you can encrypt a single block.

Option B: The "Smart" Peripheral (On-the-Fly)

- **How it works:** You only send the Master Key to the accelerator. The hardware has a second circuit running alongside the main loop that calculates the next key *while* the encryption is happening.
- **Pros:** Extremely fast. The CPU just sends one key and says "Go."
- **Cons:** You have to design and build the Key Expansion logic (which looks very similar to the SubBytes logic) on your chip.

For your project, do you want to prioritize simplicity (Option A) or maximum performance (Option B)?

Option B. But I have a question. Why do we just xor each bit in the block with the key? This isn't exactly like adding? so why is XOR = add?

Why XOR = Add?

That is a fantastic question. It feels wrong because in the "real world," $\$1 + 1 = 2\$$. But in the binary world of AES (Galois Field $\$GF(2)\$$), the rules are different.

In this math, we are doing addition without carry.

Think about adding two single bits:

- $\$0 + 0 = 0\$$
- $\$0 + 1 = 1\$$
- $\$1 + 0 = 1\$$
- $\$1 + 1 = 0\$$ (In normal math this is 2, or 10 binary. But here, we drop the carry bit, so it wraps back to 0).

Now look at the truth table for an XOR gate:

- $0 \text{ XOR } 0 = 0$
- $0 \text{ XOR } 1 = 1$
- $1 \text{ XOR } 0 = 1$
- $1 \text{ XOR } 1 = 0$

They are identical! That is why, in cryptography and finite field math, whenever you see a "Plus" sign ($\$+\$$), you should physically wire up an XOR gate.

Option B: The "Smart" Accelerator (On-the-Fly Key Gen)

You chose the high-performance route! This is the "professional" way to build hardware.

Instead of asking the RISC-V processor to compute 11 keys and load them (which takes time), your hardware will take the initial "Master Key" and generate the next Round Key *while* the encryption is happening.

We call this module the Key Expander. It sits right next to your main Encryption datapath.

How Key Expansion Works

The Key Expander generates the next 128-bit key based on the previous 128-bit key. It's simpler than the main encryption loop, but it has a specific "twisting" step.

The 128-bit key is split into 4 words (columns): $\$w_0, w_1, w_2, w_3\$$.

To get the new set of words (w'_0, w'_1, w'_2, w'_3), we follow a cascade:

1. The "Twist" (New Column 0):

This is the hardest part. To calculate the new first column (w'_0), we take the *last* column of the old key (w_3) and modify it:

- RotWord: Shift the bytes up (cyclic shift).
- SubWord: Run the bytes through the S-Box (yes, the same table we used before!).
- Rcon: XOR the top byte with a "Round Constant" (a small number that changes each round to prevent symmetry).
- Final XOR: XOR that result with the old w_0 .

2. The Easy Ones (Columns 1, 2, 3):

Once you have the difficult new first column (w'_0), the rest are just a chain reaction of simple XORs:

- $w'_1 = w'_0 \oplus w_1$
- $w'_2 = w'_1 \oplus w_2$
- $w'_3 = w'_2 \oplus w_3$

Hardware Implication: 4 Extra S-Boxes

Since you chose Option B, you have a slight resource cost to pay.

The "Twist" step requires running 4 bytes through the S-Box.

- Main Encryption Loop: Uses 16 S-Boxes simultaneously.
- Key Expander: Needs 4 S-Boxes simultaneously.

If you want to do this all in one clock cycle, you cannot share them. You need to instantiate 4 extra copies of the S-Box table for the Key Expander.

- Total Memory: $16 \times (\text{Data}) + 4 \times (\text{Key}) = 20 \times (\text{copies})$.
- Total Size: $20 \times 256 \times (\text{bytes}) = 5 \times (\text{KB})$.

Is 5 KB still acceptable for your memory budget?

The Final Architecture

If that memory usage is okay, here is the complete picture of your peripheral:

1. Registers:

- **KEY_REG**: 128-bit write-only register (from RISC-V).
- **DATA_IN**: 128-bit write-only register (from RISC-V).
- **DATA_OUT**: 128-bit read-only register (to RISC-V).
- **STATUS**: 1-bit flag ("Busy/Done").

2. Logic:

- Cycle 0: You load **DATA_IN** and **KEY_REG**.

- Cycles 1–9:
 - The Encryption Engine scrambles the data using the current Key.
 - The Key Expander calculates the *next* Key in parallel.
- Cycle 10: Final round (skip MixColumns).
- Cycle 11: The **STATUS** flag goes to "Done".

Does this "parallel twin engine" approach make sense to you? If so, we can move on to how you actually connect this to the RISC-V bus (like AXI or TileLink).

Explain twist in more detail

The "Twist" (often called the SgS -function in textbooks) is the most complex part of the Key Expander. It is the special sauce that prevents the key schedule from being a simple, predictable pattern.

If we didn't have this twist, every column of the key would just be the previous column XOR'd with something else. That kind of linearity is weak. The twist injects non-linearity (scrambling) into the key generation process.

Let's break down exactly what happens to that last column (w_3) to turn it into the magical value we need.

The input is a 4-byte column (32 bits). Let's imagine our input column w_3 contains these 4 bytes (from top to bottom):

[A, B, C, D]

The "Twist" applies three operations in this specific order:

1. RotWord (Rotate Word)

This is a simple cyclic shift of the bytes. We take the top byte and move it to the very bottom.

- Input: [A, B, C, D]
- Output: [B, C, D, A]

Hardware: This is free. It's just wires! You wire the output of byte 0 to the input of byte 3, and shift the others up.

2. SubWord (Substitute Word)

This is where we use the S-Boxes. We take each of the 4 bytes from the result of the rotation and run them individually through the S-Box table—the exact same table you used for the main encryption loop.

- Input: [B, C, D, A]
- Action: Look up $\text{S-Box}(B)$, $\text{S-Box}(C)$, etc.
- Output: [B', C', D', A'] (where the prime ' symbol means "substituted value")

Hardware: This is why you need those 4 extra S-Box copies. You are processing 4 bytes at once here, while your main encryption engine is simultaneously processing 16 bytes elsewhere. If you tried to share the S-Boxes, you'd have a traffic jam (resource conflict).

3. XOR with Rcon (Round Constant)

This is the final touch. We take the result from the S-Boxes and XOR it with a "Round Constant" (\$Rcon\$).

The \$Rcon\$ is a special value that changes for every round.

- It is a 4-byte column, but the bottom 3 bytes are always 0.
- Only the top byte changes.

The values for the top byte follow the power of 2 in the Galois Field:

- Round 1: 01
- Round 2: 02
- Round 3: 04
- Round 4: 08
- Round 5: 10 ... and so on. (Note: after 0x80, it wraps around using the 0x1B\$ trick we discussed).

The Operation:

You only XOR the top byte (\$B'\$) with the Rcon value. The other three bytes (\$C'\$, \$D'\$, \$A'\$) are XOR'd with 0 (which means they don't change).

- Input: [\$B'\$, \$C'\$, \$D'\$, \$A'\$]
- Action: \$B'\$ XOR \$Rcon[round_number]\$
- Output: The final "Twisted" column.

Hardware: You can implement the Rcon values as a tiny lookup table (a generic case statement in Verilog/VHDL) that takes the Round Counter (1-10) as input and spits out the correct 8-bit constant.

Putting it all together

So, to generate the first column of the *new* key (\$w'_0\$), the formula is:

$$w'_0 = w_0 \oplus \underbrace{(\text{SubWord}(\text{RotWord}(w_3)) \oplus Rcon)}_{\text{The Twist}}$$

1. Take the old last column (\$w_3\$).
2. Rotate it.
3. Substitute it (S-Box).
4. XOR the top byte with the Round Constant.
5. Take that result and XOR it with the old first column (\$w_0\$).

Why do we do this?

This ensures that a small difference in the Key doesn't just "slide" through the rounds.

The rotation moves differences to new rows, the S-Box explodes small differences into big ones, and the Rcon ensures that Round 1 looks mathematically different from Round 2.

Does this breakdown clarify why we need those 4 extra S-Boxes?

Yes

This is a huge milestone. You now have the complete schematic for the "datapath"—the muscles of your accelerator.

You have:

1. The Encryption Engine: 16 parallel S-Boxes + MixColumns logic.
2. The Key Expander: 4 parallel S-Boxes + Rot/Rcon logic.

Now we need to add the "brain" to control these muscles. We need to build the Control Unit and define how your RISC-V processor talks to it.

1. The Control Unit (Finite State Machine)

You need a simple state machine (FSM) to coordinate the rounds. Since you chose the "On-the-Fly" key expansion, the timing is critical. Everything happens in lockstep.

Here is the lifecycle of one encryption operation:

- IDLE State: The hardware does nothing. It waits for the RISC-V processor to set a "Start" signal.
- LOAD State: You copy the **DATA_IN** and **KEY** registers into internal registers. (Crucial: You must capture the key now because the Key Expander is about to destroy the original key to create the next one).
- ROUNDS 1–9:
 - Datapath: Performs SubBytes $\$ \backslash to \$$ ShiftRows $\$ \backslash to \$$ MixColumns $\$ \backslash to \$$ AddRoundKey.
 - Key Unit: Calculates the *next* round key.
 - Counter: Increments 1, 2, ... 9.
- ROUND 10 (Final):
 - Datapath: Performs SubBytes $\$ \backslash to \$$ ShiftRows $\$ \backslash to \$$ ~~MixColumns~~ $\$ \backslash to \$$ AddRoundKey. (Note the skipped step!).
- DONE State:
 - The result is written to the **DATA_OUT** register.
 - The **DONE** flag is raised so the RISC-V knows it can read the answer.

2. The Programmer's View (Register Map)

To your RISC-V core, this accelerator just looks like a specific memory address (Memory Mapped IO).

Assuming you are using a 32-bit RISC-V processor, here is a standard way to map the registers. You will likely connect this to a bus like AXI4-Lite, APB, or TileLink.

Base Address: **0x1000_0000** (Example)

Offset	Name	Type	Description
0x00	CTRL	RW	Control/Status. Bit 0: START (Write 1 to begin) Bit 1: DONE (Read 1 when finished)
0x04	KEY_0	W	Key bits [31:0]
0x08	KEY_1	W	Key bits
0x0C	KEY_2	W	Key bits
0x10	KEY_3	W	Key bits
0x14	TEXT_0	W	Input Data [31:0]
0x18	TEXT_1	W	Input Data
0x1C	TEXT_2	W	Input Data

0x20	TEXT_3	W	Input Data
0x24	RESULT_0	R	Output Ciphertext [31:0]
0x28	RESULT_1	R	Output Ciphertext
0x2C	RESULT_2	R	Output Ciphertext
0x30	RESULT_3	R	Output Ciphertext

A Critical Design Choice: Decryption

We have talked exclusively about Encryption (turning Plaintext to Ciphertext).

However, AES Decryption is not simply running the machine in reverse.

- Inverse S-Box: You need a *different* lookup table to reverse the substitution.
- Inverse MixColumns: You need a *different* matrix multiplication (multiplying by 9, 11, 13, 14 instead of 2, 3, 1, 1).

The Cost: Adding decryption logic will roughly double your area usage because you need a second set of tables (Inverse S-Boxes) and more complex math logic.

For your first RISC-V peripheral project, do you want to:

1. Build Encryption-Only: Keep it simple and small. (Common for IoT devices that only need to *send* secure data).
2. Build Both: A full-featured crypto engine, accepting the extra hardware complexity.

encryption only. Give me the state machine

Designing the Finite State Machine (FSM) for an "Encryption-Only" accelerator is much cleaner because the data flows in only one direction.

Since you are using On-the-Fly Key Expansion, your FSM needs to coordinate two parallel processes: transforming the data and updating the key for the *next* round.

Here is the hardware state machine design.

The 5 Critical States

For AES-128, we need to handle the "Initial Round" (which is just an XOR), the 9 main rounds, and the final round.

1. S_IDLE

- Action: Wait for the RISC-V processor to send the **START** signal.
- Outputs: **DONE = 0**, **BUSY = 0**.
- Transition: If **START == 1**, go to **S_INIT**.

2. S_INIT (Clock Cycle 0)

- Action:
 - Load **DATA_IN** from the bus into the State Register.
 - Load **KEY_IN** from the bus into the Key Register.
 - The "Round 0" Operation: Perform the initial **AddRoundKey** immediately. Since this is just XOR, we can do **State = DATA_IN ^ KEY_IN** in this same cycle.
- Counter: Reset Round Counter to **1**.
- Transition: Go to **S_MAIN**.

3. S_MAIN (Clock Cycles 1–9)

- Action: Execute the standard AES Round.
 - Datapath: Apply **SubBytes** $\rightarrow$ **ShiftRows** $\rightarrow$ **MixColumns** $\rightarrow$ **AddRoundKey**.
 - Key Expander: Calculate the *next* round key simultaneously.
- Counter: Increment Round Counter ($r = r + 1$).
- Transition:
 - If **Counter < 10**, stay in **S_MAIN**.
 - If **Counter == 10**, go to **S_FINAL**.

4. S_FINAL (Clock Cycle 10)

- Action: Execute the special final round.
 - Datapath: Apply **SubBytes** \$to\$ **ShiftRows** \$to\$ (SKIP MixColumns) \$to\$ **AddRoundKey**.
 - Key Expander: Calculate the final round key.
- Transition: Go to **S_DONE**.

5. S_DONE

- Action: Hold the result in the **DATA_OUT** registers so the processor can read it.
- Outputs: **DONE = 1**, **BUSY = 0**.
- Transition: Wait for the processor to clear the **START** signal or acknowledge the read, then return to **S_IDLE**.

The Control Logic Table

This is how you wire your FSM outputs to the datapath logic. You likely need a few multiplexers (Muxes) to control the flow.

State	Round Counter	Key Update	Datapath Input	MixColumns?
IDLE	0	Hold	X	X
INIT	0	Load New	DATA_IN	X
MAIN	1–9	Generate Next	State_Reg	Enable
FINAL	10	Generate Next	State_Reg	Bypass
DONE	0	Hold	State_Reg	X

Implementation Detail: The "Bypass" Signal

The biggest difference between **S_MAIN** and **S_FINAL** is the MixColumns step.

You don't need two separate circuits. You just need a Mux (Multiplexer) after the

MixColumns block.

- Input A: The output of MixColumns.
- Input B: The input that *went into* MixColumns (bypassing it).
- Selector: **Is_Final_Round** (driven by the FSM).

Verilog Logic (Pseudocode)

Here is how this logic looks in hardware description language:

Verilog

```
always @(posedge clk) begin
  if (reset) begin
    state <= IDLE;
    round_counter <= 0;
  end else begin
    case (state)
      IDLE: begin
        done <= 0;
        if (start) state <= INIT;
      end

      INIT: begin
        // Load registers and perform Initial XOR
        state_reg <= data_in ^ key_in;
        key_reg <= key_in;
        round_counter <= 1;
        state <= MAIN;
      end

      MAIN: begin
        // 1. Update State (Full Round)
        state_reg <= AddRoundKey(MixColumns(ShiftRows(SubBytes(state_reg))),
next_key);
        // 2. Update Key
        key_reg <= next_key;

        round_counter <= round_counter + 1;

        if (round_counter == 9)
          state <= FINAL;
      end

      FINAL: begin
```

```

        // Update State (Skip MixColumns)
        state_reg <= AddRoundKey(ShiftRows(SubBytes(state_reg)), next_key);
        done <= 1;
        state <= DONE;
    end

    DONE: begin
        if (!start) state <= IDLE; // Wait for handshake
    end
endcase
end
end
end

```

Next Step

This FSM provides a latency of exactly 11 clock cycles per 128-bit block.

- At 50 MHz (common for simple FPGA projects), that's roughly 580 Mbps throughput.
- The software implementation on a standard RISC-V core would likely take hundreds of cycles.

You now have the Datapath (S-Boxes/XORs) and the Controller (FSM).

Are you ready to move on to the System Verilog / Verilog code structure to start implementing this, or do you want to verify the MixColumns logic implementation first?

If my data bus is 32 bits. how can i output and input the data?

This is a classic hardware design problem: Width Mismatch.

Your AES core needs 128 bits all at once to start working, but your RISC-V processor can only deliver 32 bits at a time. It's like trying to fill a bucket using a teacup.

You solve this by creating a Buffer (Register Bank) inside your peripheral. You don't feed the data directly into the AES logic; you park it in a holding area first.

The Strategy: "Pack and Unpack"

You will need four 32-bit write operations to assemble one full 128-bit AES block.

1. Input (RISC-V $\rightarrow$ Accelerator)

You view the 128-bit input register **DATA_IN** as four separate 32-bit slices.

- Write 1 (Address **0x14**): Fills the bottom slice (**bits 31:0**).
- Write 2 (Address **0x18**): Fills the next slice (**bits 63:32**).
- Write 3 (Address **0x1C**): Fills the next slice (**bits 95:64**).
- Write 4 (Address **0x20**): Fills the top slice (**bits 127:96**).

Only *after* the processor has written all four chunks do you trigger the **START** signal. The AES core then reads all 128 bits from this buffer instantly in parallel.

2. Output (Accelerator $\rightarrow$ RISC-V)

When the AES core finishes, it dumps the 128-bit result into an output buffer **DATA_OUT**. The processor then reads it out in four chunks.

- Read 1 (Address **0x24**): Reads **bits 31:0**.
- Read 2 (Address **0x28**): Reads **bits 63:32**.
- ...and so on.

The Hardware Logic (Verilog/SystemVerilog)

Here is how you actually code this interface. You use the Address bits to route the 32-bit data to the correct slice of the 128-bit register.

Assume **P_WDATA** is your 32-bit data from the processor, and **P_ADDR** is the address.

Verilog

```
reg [127:0] data_in_buffer; // The internal 128-bit holding tank
```

```
// Write Logic (Filling the bucket)
```

```
always @(posedge clk) begin
```

```
    if (write_enable) begin
```

```
        case (P_ADDR[5:0]) // Look at the lower address bits
```

```

        6'h14: data_in_buffer[31:0]  <= P_WDATA; // Fill bottom chunk
        6'h18: data_in_buffer[63:32] <= P_WDATA; // Fill chunk 2
        6'h1C: data_in_buffer[95:64] <= P_WDATA; // Fill chunk 3
        6'h20: data_in_buffer[127:96] <= P_WDATA; // Fill top chunk
    endcase
end
end

```

The "Endianness" Trap (Watch out!)

This is where many hardware designers get stuck for days.

- RISC-V is usually Little Endian (Least Significant Byte is at the lowest address).
- AES treats data as a stream of bytes.

When you write **0xAABBCCDD** to your register, you need to be absolutely sure that **0xDD** ends up in the "Byte 0" slot of your AES state grid, and not **0xAA**.

Rule of Thumb:

If your test vectors (standard inputs/outputs you use to check your design) are failing, swap the byte order in your 32-bit writes.

- Instead of **data_in_buffer[31:0] <= P_WDATA;**
- You might need **data_in_buffer[31:0] <= {P_WDATA[7:0], P_WDATA[15:8], P_WDATA[23:16], P_WDATA[31:24]};**

Would you like to review the standard NIST AES Test Vectors next? You will need these "gold standard" inputs and outputs to verify your accelerator is working correctly.

```

module AES_key_engine (
    input clk,
    input reset,
    input [3:0] key_engine_counter,
    input [127:0] original_key,
    output reg [127:0] round_key,
    output reg round_key_rdy
);

reg [127:0] key_schedule [0:10]; // Array to hold round keys

always@(clk or reset) begin
    if (reset) begin
        round_key <= 128'b0;
        round_key_rdy <= 1'b0;
    end else begin
        case (key_engine_counter)
            4'd0: begin
                round_key <= 128'b0;
                round_key_rdy <= 1'b0;
            end
            4'd1: begin
                // calculate all round keys
                //insert key expansion logic here
                round_key <= 128'b0;
                round_key_rdy <= 1'b1;
            end
            4'd2: begin
                // Key expansion logic for round 0(addRoundKey)
                round_key <= key_schedule[0];
            end
            4'd3: begin
                // Key expansion logic for round 1
                round_key <= key_schedule[1];
            end
            4'd4: begin
                // Key expansion logic for round 2
                round_key <= key_schedule[2];
            end
            4'd5: begin
                // Key expansion logic for round 3
                round_key <= key_schedule[3];
            end
        end
    end
end

```

```

4'd6: begin
    // Key expansion logic for round 4
    round_key <= key_schedule[4];
end
4'd7: begin
    // Key expansion logic for round 5
    round_key <= key_schedule[5];
end
4'd8: begin
    // Key expansion logic for round 6
    round_key <= key_schedule[6];
end
4'd9: begin
    // Key expansion logic for round 7
    round_key <= key_schedule[7];
end
4'd10: begin
    // Key expansion logic for round 8
    round_key <= key_schedule[8];
end
4'd11: begin
    // Key expansion logic for round 9
    round_key <= key_schedule[9];
end
4'd12: begin
    // Key expansion logic for round 10
    round_key <= key_schedule[10];
end
default: begin
    round_key <= 128'b0;
    round_key_rdy <= 1'b0;
end
endcase
end
end

endmodule

```

Give some feedback on this? this is the key expansion module

This code is a great start, but it reveals a conflict between the Option B (On-the-Fly) approach we discussed and the code you actually wrote.

You have implemented Option A (Storage/Lookup), but with a critical timing flaw.

Here are the three specific issues to fix:

1. The "One Cycle" Impossibility

In `case 4'd1`, you have a comment: `// calculate all round keys`.

Hardware Reality Check: You cannot calculate all 10 rounds of keys in a single clock cycle.

- Round 10 depends on Round 9, which depends on Round 8... etc.
- To do this in one cycle, the signal would have to pass through the S-Box logic 40 times. This would create a massive critical path, forcing your clock speed to be extremely slow (likely < 10 MHz).

2. Massive Area Cost

The line `reg [127:0] key_schedule [0:10];` creates 1,408 registers (flip-flops).

- Option B (On-the-Fly) only needs 128 registers.
- By pre-allocating the whole array, you are wasting silicon area. The goal of "On-the-Fly" is to delete the old key as soon as you generate the new one.

3. Syntax Issue

`always@(clk or reset)` is old-school (Verilog-95) and can sometimes infer latches if you aren't careful.

- Fix: Use `always @(posedge clk or posedge reset)` to explicitly define edge-triggered sequential logic.

The Fix: True "On-the-Fly" Implementation

To switch this back to the high-performance Option B, we need to remove the array. Instead, we treat the key as a living variable that changes every clock cycle.

Here is the corrected logic. Notice there is no array, just one `current_key` register.

Verilog

```
module AES_key_engine (  
    input wire clk,  
    input wire reset,
```

```

input wire load_key,      // Signal to load the initial Master Key
input wire enable_update, // Signal to generate the NEXT key
input wire [3:0] round_count, // Current round (used for Rcon lookup)
input wire [127:0] key_in, // The Master Key (from RISC-V)
output reg [127:0] current_key // The active key for this round
);

```

```

// -----
// 1. Define the wires for the "Next Key" calculation
// -----

```

```

wire [31:0] w0, w1, w2, w3; // The old columns
wire [31:0] w3_twisted;     // The result of Rot/Sub/Rcon

```

```

// Break the current 128-bit key into 32-bit columns
assign w0 = current_key[127:96];
assign w1 = current_key[95:64];
assign w2 = current_key[63:32];
assign w3 = current_key[31:0];

```

```

// -----
// 2. The "Twist" (g-function) Logic
// -----

```

```

// We need 4 S-Boxes for the Key Expander.
// Ideally, instantiate a module, but here is the logic flow:

```

```

wire [31:0] rot_w3;
wire [31:0] sub_w3;
wire [31:0] rcon_w3;

```

```

// A. RotWord (Cyclic shift left 8 bits)
assign rot_w3 = {w3[23:0], w3[31:24]};

```

```

// B. SubWord (S-Box Lookup)
// You must instantiate 4 S-Box lookups here.
// For now, let's assume a module 'sbox_lookup' exists.
sbox_lookup s0 (.addr(rot_w3[31:24]), .out(sub_w3[31:24]));
sbox_lookup s1 (.addr(rot_w3[23:16]), .out(sub_w3[23:16]));
sbox_lookup s2 (.addr(rot_w3[15:08]), .out(sub_w3[15:08]));
sbox_lookup s3 (.addr(rot_w3[07:00]), .out(sub_w3[07:00]));

```

```

// C. XOR with Rcon (Round Constant)
// We need a small function to get Rcon based on round_count
function [7:0] get_rcon;

```



```

input [3:0] round;
case(round)
  4'd1: get_rcon = 8'h01;
  4'd2: get_rcon = 8'h02;
  4'd3: get_rcon = 8'h04;
  4'd4: get_rcon = 8'h08;
  4'd5: get_rcon = 8'h10;
  4'd6: get_rcon = 8'h20;
  4'd7: get_rcon = 8'h40;
  4'd8: get_rcon = 8'h80;
  4'd9: get_rcon = 8'h1B;
  4'd10: get_rcon = 8'h36;
  default: get_rcon = 8'h00;
endcase
endfunction

// Only XOR the top byte with Rcon
assign w3_twisted = sub_w3 ^ {get_rcon(round_count), 24'b0};

// -----
// 3. The Chain Reaction (XORs to make the new key)
// -----
wire [31:0] new_w0, new_w1, new_w2, new_w3;

assign new_w0 = w0 ^ w3_twisted;
assign new_w1 = w1 ^ new_w0;
assign new_w2 = w2 ^ new_w1;
assign new_w3 = w3 ^ new_w2;

// -----
// 4. The Sequential Block (Update Register)
// -----
always @(posedge clk or posedge reset) begin
  if (reset) begin
    current_key <= 128'b0;
  end else if (load_key) begin
    // Initial Load: Just grab the master key
    current_key <= key_in;
  end else if (enable_update) begin
    // Update: Replace current key with the calculated next key
    current_key <= {new_w0, new_w1, new_w2, new_w3};
  end
end
end

```

endmodule

Why this is better for you:

1. **Combinational Logic:** The variables `new_w0` etc. are calculated instantly by gates (wires).
2. **Clock Edge:** When the clock ticks (and `enable_update` is high), the `current_key` register simply captures that new value.
3. **Synchronization:** This runs perfectly in sync with your main state machine. When your main FSM moves from Round 1 to Round 2, it pulses `enable_update`, and the Key Engine instantly prepares the key for Round 2.

Do you see how we replaced the `case` statement (lookup) with the `assign` statements (math logic)? This is the essence of "On-the-Fly."

Can you write the sbbox lut in verilog?

Here is the Verilog code for the S-Box.

I have implemented this using a **case** statement. This is the safest way to ensure your FPGA synthesizer implements this as Combinational Logic (LUTs) rather than trying to infer a synchronous Block RAM (which would break your timing).

sbbox_lookup.v

Verilog

```
module sbbox_lookup (  
    input wire [7:0] addr,  
    output reg [7:0] out  
);
```

```
// The Sensitivity List @(*) means this block updates INSTANTLY
```

```
// whenever 'addr' changes. It behaves like a complex gate.
```

```
always @(*) begin
```

```
    case (addr)
```

```
        // Row 0
```

```
        8'h00: out = 8'h63; 8'h01: out = 8'h7c; 8'h02: out = 8'h77; 8'h03: out = 8'h7b;  
        8'h04: out = 8'hf2; 8'h05: out = 8'h6b; 8'h06: out = 8'h6f; 8'h07: out = 8'hc5;  
        8'h08: out = 8'h30; 8'h09: out = 8'h01; 8'h0a: out = 8'h67; 8'h0b: out = 8'h2b;  
        8'h0c: out = 8'hfe; 8'h0d: out = 8'hd7; 8'h0e: out = 8'hab; 8'h0f: out = 8'h76;
```

```
        // Row 1
```

```
        8'h10: out = 8'hca; 8'h11: out = 8'h82; 8'h12: out = 8'hc9; 8'h13: out = 8'h7d;  
        8'h14: out = 8'hfa; 8'h15: out = 8'h59; 8'h16: out = 8'h47; 8'h17: out = 8'hf0;  
        8'h18: out = 8'had; 8'h19: out = 8'hd4; 8'h1a: out = 8'ha2; 8'h1b: out = 8'haf;  
        8'h1c: out = 8'h9c; 8'h1d: out = 8'ha4; 8'h1e: out = 8'h72; 8'h1f: out = 8'hc0;
```

```
        // Row 2
```

```
        8'h20: out = 8'hb7; 8'h21: out = 8'hfd; 8'h22: out = 8'h93; 8'h23: out = 8'h26;  
        8'h24: out = 8'h36; 8'h25: out = 8'h3f; 8'h26: out = 8'hf7; 8'h27: out = 8'hcc;  
        8'h28: out = 8'h34; 8'h29: out = 8'ha5; 8'h2a: out = 8'he5; 8'h2b: out = 8'hf1;  
        8'h2c: out = 8'h71; 8'h2d: out = 8'hd8; 8'h2e: out = 8'h31; 8'h2f: out = 8'h15;
```

```
        // Row 3
```

```
        8'h30: out = 8'h04; 8'h31: out = 8'hc7; 8'h32: out = 8'h23; 8'h33: out = 8'hc3;  
        8'h34: out = 8'h18; 8'h35: out = 8'h96; 8'h36: out = 8'h05; 8'h37: out = 8'h9a;  
        8'h38: out = 8'h07; 8'h39: out = 8'h12; 8'h3a: out = 8'h80; 8'h3b: out = 8'he2;  
        8'h3c: out = 8'heb; 8'h3d: out = 8'h27; 8'h3e: out = 8'hb2; 8'h3f: out = 8'h75;
```

```
        // Row 4
```

8'h40: out = 8'h09; 8'h41: out = 8'h83; 8'h42: out = 8'h2c; 8'h43: out = 8'h1a;
8'h44: out = 8'h1b; 8'h45: out = 8'h6e; 8'h46: out = 8'h5a; 8'h47: out = 8'ha0;
8'h48: out = 8'h52; 8'h49: out = 8'h3b; 8'h4a: out = 8'hd6; 8'h4b: out = 8'hb3;
8'h4c: out = 8'h29; 8'h4d: out = 8'he3; 8'h4e: out = 8'h2f; 8'h4f: out = 8'h84;

// Row 5

8'h50: out = 8'h53; 8'h51: out = 8'hd1; 8'h52: out = 8'h00; 8'h53: out = 8'hed;
8'h54: out = 8'h20; 8'h55: out = 8'hfc; 8'h56: out = 8'hb1; 8'h57: out = 8'h5b;
8'h58: out = 8'h6a; 8'h59: out = 8'hcb; 8'h5a: out = 8'hbe; 8'h5b: out = 8'h39;
8'h5c: out = 8'h4a; 8'h5d: out = 8'h4c; 8'h5e: out = 8'h58; 8'h5f: out = 8'hcf;

// Row 6

8'h60: out = 8'hd0; 8'h61: out = 8'hef; 8'h62: out = 8'haa; 8'h63: out = 8'hfb;
8'h64: out = 8'h43; 8'h65: out = 8'h4d; 8'h66: out = 8'h33; 8'h67: out = 8'h85;
8'h68: out = 8'h45; 8'h69: out = 8'hf9; 8'h6a: out = 8'h02; 8'h6b: out = 8'h7f;
8'h6c: out = 8'h50; 8'h6d: out = 8'h3c; 8'h6e: out = 8'h9f; 8'h6f: out = 8'ha8;

// Row 7

8'h70: out = 8'h51; 8'h71: out = 8'ha3; 8'h72: out = 8'h40; 8'h73: out = 8'h8f;
8'h74: out = 8'h92; 8'h75: out = 8'h9d; 8'h76: out = 8'h38; 8'h77: out = 8'hf5;
8'h78: out = 8'hbc; 8'h79: out = 8'hb6; 8'h7a: out = 8'hda; 8'h7b: out = 8'h21;
8'h7c: out = 8'h10; 8'h7d: out = 8'hff; 8'h7e: out = 8'hf3; 8'h7f: out = 8'hd2;

// Row 8

8'h80: out = 8'hcd; 8'h81: out = 8'h0c; 8'h82: out = 8'h13; 8'h83: out = 8'hec;
8'h84: out = 8'h5f; 8'h85: out = 8'h97; 8'h86: out = 8'h44; 8'h87: out = 8'h17;
8'h88: out = 8'hc4; 8'h89: out = 8'ha7; 8'h8a: out = 8'h7e; 8'h8b: out = 8'h3d;
8'h8c: out = 8'h64; 8'h8d: out = 8'h5d; 8'h8e: out = 8'h19; 8'h8f: out = 8'h73;

// Row 9

8'h90: out = 8'h60; 8'h91: out = 8'h81; 8'h92: out = 8'h4f; 8'h93: out = 8'hdc;
8'h94: out = 8'h22; 8'h95: out = 8'h2a; 8'h96: out = 8'h90; 8'h97: out = 8'h88;
8'h98: out = 8'h46; 8'h99: out = 8'hee; 8'h9a: out = 8'hb8; 8'h9b: out = 8'h14;
8'h9c: out = 8'hde; 8'h9d: out = 8'h5e; 8'h9e: out = 8'h0b; 8'h9f: out = 8'hdb;

// Row A

8'ha0: out = 8'he0; 8'ha1: out = 8'h32; 8'ha2: out = 8'h3a; 8'ha3: out = 8'h0a;
8'ha4: out = 8'h49; 8'ha5: out = 8'h06; 8'ha6: out = 8'h24; 8'ha7: out = 8'h5c;
8'ha8: out = 8'hc2; 8'ha9: out = 8'hd3; 8'haa: out = 8'hac; 8'hab: out = 8'h62;
8'hac: out = 8'h91; 8'had: out = 8'h95; 8'hae: out = 8'he4; 8'haf: out = 8'h79;

// Row B

8'hb0: out = 8'he7; 8'hb1: out = 8'hc8; 8'hb2: out = 8'h37; 8'hb3: out = 8'h6d;

```
8'hb4: out = 8'h8d; 8'hb5: out = 8'hd5; 8'hb6: out = 8'h4e; 8'hb7: out = 8'ha9;  
8'hb8: out = 8'h6c; 8'hb9: out = 8'h56; 8'hba: out = 8'hf4; 8'hbb: out = 8'hea;  
8'hbc: out = 8'h65; 8'hbd: out = 8'h7a; 8'hbe: out = 8'hae; 8'hbf: out = 8'h08;
```

```
// Row C
```

```
8'hc0: out = 8'hba; 8'hc1: out = 8'h78; 8'hc2: out = 8'h25; 8'hc3: out = 8'h2e;  
8'hc4: out = 8'h1c; 8'hc5: out = 8'ha6; 8'hc6: out = 8'hb4; 8'hc7: out = 8'hc6;  
8'hc8: out = 8'he8; 8'hc9: out = 8'hdd; 8'hca: out = 8'h74; 8'hcb: out = 8'h1f;  
8'hcc: out = 8'h4b; 8'hcd: out = 8'hbd; 8'hce: out = 8'h8b; 8'hcf: out = 8'h8a;
```

```
// Row D
```

```
8'hd0: out = 8'h70; 8'hd1: out = 8'h3e; 8'hd2: out = 8'hb5; 8'hd3: out = 8'h66;  
8'hd4: out = 8'h48; 8'hd5: out = 8'h03; 8'hd6: out = 8'hf6; 8'hd7: out = 8'h0e;  
8'hd8: out = 8'h61; 8'hd9: out = 8'h35; 8'hda: out = 8'h57; 8'hdb: out = 8'hb9;  
8'hdc: out = 8'h86; 8'hdd: out = 8'hc1; 8'hde: out = 8'h1d; 8'hdf: out = 8'h9e;
```

```
// Row E
```

```
8'he0: out = 8'he1; 8'he1: out = 8'hf8; 8'he2: out = 8'h98; 8'he3: out = 8'h11;  
8'he4: out = 8'h69; 8'he5: out = 8'hd9; 8'he6: out = 8'h8e; 8'he7: out = 8'h94;  
8'he8: out = 8'h9b; 8'he9: out = 8'h1e; 8'hea: out = 8'h87; 8'heb: out = 8'he9;  
8'hec: out = 8'hce; 8'hed: out = 8'h55; 8'hee: out = 8'h28; 8'hef: out = 8'hdf;
```

```
// Row F
```

```
8'hf0: out = 8'h8c; 8'hf1: out = 8'ha1; 8'hf2: out = 8'h89; 8'hf3: out = 8'h0d;  
8'hf4: out = 8'hbf; 8'hf5: out = 8'he6; 8'hf6: out = 8'h42; 8'hf7: out = 8'h68;  
8'hf8: out = 8'h41; 8'hf9: out = 8'h99; 8'hfa: out = 8'h2d; 8'hfb: out = 8'h0f;  
8'hfc: out = 8'hb0; 8'hfd: out = 8'h54; 8'hfe: out = 8'hbb; 8'fff: out = 8'h16;
```

```
default: out = 8'h00; // Safety
```

```
endcase
```

```
end
```

```
endmodule
```

How to use this module

In your [AES_key_engine](#) or [AES_datapath](#), you instantiate this module just like any other component.

For the Key Engine, you need 4 copies:

Verilog

```
sbox_lookup s0 (.addr(rot_w3[31:24]), .out(sub_w3[31:24]));  
sbox_lookup s1 (.addr(rot_w3[23:16]), .out(sub_w3[23:16]));
```

// ... etc

For the Main Datapath (SubBytes step), you will need 16 copies.

Would you like to see the Verilog Loop trick to instantiate 16 copies without writing 16 lines of code?

in last row

```
8'hfc: out = 8'hb0; 8'hfd: out = 8'h54; 8'hfe: out = 8'hbb; 8'fff: out = 8'h16;
```

is 8'fff? valid

You have a very sharp eye! That is a typo in my code.

No, 8'fff is not valid Verilog.

There are two errors in that single token:

1. Missing Base: It is missing the **h** for hexadecimal (e.g., 8'h...).
2. Too Wide: It has three **fs** (12 bits), but the size is specified as **8**.

The correct line for the last entry (index 255) should be:

Verilog

```
8'hff: out = 8'h16;
```

The "Verilog Loop" Trick (generate)

Since you now have a working S-Box module, you definitely don't want to manually type out 16 instances of it for your main encryption loop. That is messy and prone to wiring errors.

In Verilog, we use a **generate** block to create copies of hardware in a loop.

Here is how you implement the SubBytes step for the full 128-bit state using your new **sbox_lookup** module:

Verilog

```
// Assume 'state_current' is your 128-bit input
// Assume 'state_after_sub' is your 128-bit output
```

```
genvar i; // Special variable for generate loops
generate
  for (i = 0; i < 16; i = i + 1) begin : sbox_array
    // Instantiate the S-Box
    // We slice the 128-bit vector into 8-bit chunks
    sbox_lookup s_inst (
      .addr( state_current[ (i*8)+7 : (i*8) ] ),
      .out ( state_after_sub[ (i*8)+7 : (i*8) ] )
    );
  end
```

endgenerate

Why this is powerful

When you synthesize this code, the tools will physically create 16 separate copies of the logic in parallel.

- `i=0` creates an S-Box for bits `[7:0]`.
- `i=1` creates an S-Box for bits `[15:8]`.
- ...and so on.

This keeps your code clean while ensuring the hardware remains massively parallel.

Now that we have the SubBytes hardware ready, shall we look at how to implement ShiftRows using just wires (no logic)?

is RotWord in AES a cyclic right shift?

No, RotWord in AES is a cyclic left shift.

It takes a four-byte word and shifts the bytes to the left by one position. The byte that "falls off" the left side wraps around to the right side.

How it works

If you have a 32-bit word consisting of 4 bytes: $[b_0, b_1, b_2, b_3]$

RotWord transforms it into: $[b_1, b_2, b_3, b_0]$

Example

Input Word (Hex):

1A 2B 3C 4D

Apply RotWord (Left Shift):

2B 3C 4D 1A

Purpose in AES

RotWord is strictly used during the Key Expansion (Key Schedule) phase, specifically when generating new columns for the round keys. It is never used during the standard encryption rounds (like SubBytes or MixColumns).

Would you like to see how this integrates with the SubWord step that usually follows it?

```

module AES_encryption_engine(
    input clk,
    input reset,
    input [4:0] encryption_engine_count,
    input [127:0] data_in,
    input [127:0] round_key,
    output reg [127:0] data_out
);

localparam
    IDLE      = 3'b000,
    ADD_ROUND_KEY = 3'b001,
    ROUND_FUNC  = 3'b010,
    FINAL_ROUND = 3'b011;

reg [2:0] current_state, next_state;

wire [127:0] substituted_data;

always @(posedge clk or posedge reset) begin
    if (reset) begin
        current_state <= IDLE;
    end else begin

```

```

        current_state <= next_state;
    end

end

sbox_lookup sbox0 (
    .addr(),
    .out (substituted_word[31:24])
);

always @(*) begin
    data_out = 128'b0;
    next_state = IDLE;
    case (current_state)
        IDLE: begin
            if (encryption_engine_count == 5'd0) begin
                next_state = ADD_ROUND_KEY;
            end else begin
                next_state = IDLE;
            end
        end
    end
end

ADD_ROUND_KEY: begin
    data_out = data_in ^ round_key;
    next_state = ROUND_FUNC;
end

```

end

ROUND_FUNC: begin

// Placeholder for actual round function (SubBytes, ShiftRows, MixColumns)

data_out = data_in ^ round_key; // Simplified for illustration

if (encryption_engine_count == 5'd9) begin

next_state = FINAL_ROUND;

end else begin

next_state = ADD_ROUND_KEY;

end

end

FINAL_ROUND: begin

// Placeholder for final round function (SubBytes, ShiftRows)

data_out = data_in ^ round_key; // Simplified for illustration

next_state = IDLE;

end

default: begin

next_state = IDLE;

end

endcase

end

Help with this code?

This code has the right idea but needs a structural shift to work with the Key Engine you built previously.

The Logic Problem

Your Key Engine generates one key per clock cycle. Your Encryption Engine currently uses an internal State Machine (**IDLE** -> **ADD** -> **FUNC**), which takes multiple clock cycles per round. This will cause the two engines to fall out of sync immediately.

The Solution: Pipeline It

Instead of an internal FSM, use the **encryption_engine_count** (which presumably tracks rounds 0-10) to control the flow directly. The architecture should be a simple loop: Register -> SubBytes -> ShiftRows -> MixColumns -> Register.

Here is the corrected, complete module. I have added the 16 S-Box instantiations, the ShiftRows logic, and the MixColumns logic.

Corrected Verilog Code

Verilog

```
module AES_encryption_engine(
    input clk,
    input reset,
    input [3:0] round_count, // Connects to your counter (0 to 10)
    input [127:0] plaintext, // Renamed from data_in for clarity
    input [127:0] round_key, // From AES_key_engine
    output reg [127:0] ciphertext
);

    reg [127:0] state; // The intermediate cipher state
    wire [127:0] sb_out; // After SubBytes
    wire [127:0] sr_out; // After ShiftRows
    wire [127:0] mc_out; // After MixColumns
```

```
wire [127:0] next_state_in; // The value to store in the register
```

```
// =====
```

```
// 1. SUBBYTES (Instantiate 16 S-Boxes)
```

```
// =====
```

```
genvar i;
```

```
generate
```

```
  for (i = 0; i < 16; i = i + 1) begin : sbbox_gen
```

```
    // Assuming input [127:120] is Byte 0
```

```
    sbbox_lookup u_sbbox (
```

```
      .addr(state[127-(i*8) : 127-(i*8)-7]),
```

```
      .out (sb_out[127-(i*8) : 127-(i*8)-7])
```

```
    );
```

```
  end
```

```
endgenerate
```

```
// =====
```

```
// 2. SHIFTRROWS (Wire Mapping)
```

```
// =====
```

```
// Row 0: No shift
```

```
// Row 1: Left shift 1 byte
```

```
// Row 2: Left shift 2 bytes
```

```
// Row 3: Left shift 3 bytes
```

```
// Constructing the 128-bit wire manually to ensure correct mapping
```

```
// Bytes are indexed 0 (MSB) to 15 (LSB)
```

```
assign sr_out[127:120] = sb_out[127:120]; // B0
```

```
assign sr_out[119:112] = sb_out[ 87: 80]; // B5 (Row 1 shifted)
```

```
assign sr_out[111:104] = sb_out[ 47: 40]; // B10 (Row 2 shifted)
```

```
assign sr_out[103: 96] = sb_out[ 7: 0]; // B15 (Row 3 shifted)
```

```
assign sr_out[ 95: 88] = sb_out[ 95: 88]; // B4
```

```
assign sr_out[ 87: 80] = sb_out[ 55: 48]; // B9
```

```
assign sr_out[ 79: 72] = sb_out[ 15: 8]; // B14
```

```
assign sr_out[ 71: 64] = sb_out[103: 96]; // B3
```

```
assign sr_out[ 63: 56] = sb_out[ 63: 56]; // B8
```

```
assign sr_out[ 55: 48] = sb_out[ 23: 16]; // B13
```

```
assign sr_out[ 47: 40] = sb_out[111:104]; // B2
```

```
assign sr_out[ 39: 32] = sb_out[ 71: 64]; // B7
```

```
assign sr_out[ 31: 24] = sb_out[ 31: 24]; // B12
```

```
assign sr_out[ 23: 16] = sb_out[119:112]; // B1
```

```
assign sr_out[ 15: 8] = sb_out[ 79: 72]; // B6
```

```
assign sr_out[ 7: 0] = sb_out[ 39: 32]; // B11
```

```
// =====
```

```
// 3. MIXCOLUMNS (Function Logic)
```

```
// =====
```

```
// Helper function for Galois Field Multiply by 2
```

```
function [7:0] gm2;  
    input [7:0] x;  
    begin  
        gm2 = {x[6:0], 1'b0} ^ (x[7] ? 8'h1b : 8'h00);  
    end  
endfunction
```

```
// Helper function for Galois Field Multiply by 3 (gm2(x) ^ x)
```

```
function [7:0] gm3;  
    input [7:0] x;  
    begin  
        gm3 = gm2(x) ^ x;  
    end  
endfunction
```

```
// Mix one column (32-bit input -> 32-bit output)
```

```
function [31:0] mix_one_col;  
    input [31:0] col_in;  
    reg [7:0] b0, b1, b2, b3;  
    begin  
        b0 = col_in[31:24];
```



```

    b1 = col_in[23:16];
    b2 = col_in[15:8];
    b3 = col_in[7:0];

    mix_one_col[31:24] = gm2(b0) ^ gm3(b1) ^ b2    ^ b3;
    mix_one_col[23:16] = b0    ^ gm2(b1) ^ gm3(b2) ^ b3;
    mix_one_col[15:8]  = b0    ^ b1    ^ gm2(b2) ^ gm3(b3);
    mix_one_col[7:0]   = gm3(b0) ^ b1    ^ b2    ^ gm2(b3);

end

endfunction

// Apply MixColumns to all 4 columns
assign mc_out[127:96] = mix_one_col(sr_out[127:96]);
assign mc_out[95:64]  = mix_one_col(sr_out[95:64]);
assign mc_out[63:32]  = mix_one_col(sr_out[63:32]);
assign mc_out[31:0]   = mix_one_col(sr_out[31:0]);

// =====

// 4. MAIN REGISTER & CONTROL LOGIC

// =====

// MUX: Determine what we are calculating based on round count
// If Round 0: Plaintext
// If Round 1-9: Use MixColumns Output

```

```

// If Round 10: Bypass MixColumns (use ShiftRows Output)

assign next_state_in = (round_count == 0) ? plaintext :

    (round_count == 10) ? sr_out :

    mc_out;


always @(posedge clk or posedge reset) begin

    if (reset) begin

        state <= 128'b0;

        ciphertext <= 128'b0;

    end

    else begin

        // Operation: Load/Calculate State -> XOR with Round Key -> Store


        if (round_count == 0) begin

            // Round 0: Initial Key Addition

            state <= plaintext ^ round_key;

        end

        else if (round_count >= 1 && round_count <= 9) begin

            // Rounds 1-9: Regular Round

            // Note: We use the 'state' that was stored in the PREVIOUS clock

            // which has now flowed through Sub->Shift->Mix

            state <= mc_out ^ round_key;

        end

        else if (round_count == 10) begin

```

```

        // Round 10: Final Round (No MixColumns)

        // We do NOT update 'state' here because we are done.

        // We output directly to ciphertext.

        ciphertext <= sr_out ^ round_key;

    end

end

end

endmodule

```

Key Changes Explained

1. Removed internal FSM: The **encryption_engine_count** (which I renamed **round_count** for clarity) acts as the state.
2. Added **generate** block: You cannot effectively write AES without generating the 16 S-Box instances. This loop creates them all automatically.
3. MixColumns Logic: I added the **gm2**, **gm3**, and **mix_one_col** functions. This performs the complex mathematical matrix multiplication required by AES.
4. Final Round Logic: Notice the **if (round_count == 10)** block. AES specifies that the final round skips MixColumns. The code handles this by using **sr_out** (ShiftRows out) instead of **mc_out**.

Next Steps for you

Do you need the code for the Top Level Module that connects this engine, the key engine, and the counter together?

```
module AES_encryption_engine(  
    input clk,  
    input reset,  
    input [4:0] encryption_engine_count,  
    input [127:0] data_in,  
    input [127:0] round_key,  
    output reg [127:0] cipher_text  
);
```

That module declaration looks great! You have the clock, reset, counters, keys, and data all lined up.

Now, let's build the internals. Since this is a loop that updates every clock cycle, we need a specific place to store the "intermediate" result of the encryption as it progresses from Round 0 to Round 10.

What internal signal (a `reg`) should we define to hold this 128-bit value, and what would you like to call it?

reg [127:0] state

That is perfect. ``reg [127:0] state;`` is exactly what we need to store the intermediate result.

Now we can build the sequential logic (the ``always`` block) that updates this ``state`` on every clock cycle.

Here is the skeleton of that block:

```
``verilog
always @(posedge clk or posedge reset) begin
    if (reset) begin
        state <= 128'b0;
        cipher_text <= 128'b0;
    end else begin
        // The encryption logic goes here!
    end
end

...
```

We know we have three scenarios based on ``encryption_engine_count``:

1. ****Round 0**** (Initial Key Add)

2. ****Rounds 1-9**** (Standard Rounds)

3. ****Round 10**** (Final Round)

Let's fill in the ``else`` block one step at a time.

For ****Round 0****, we previously agreed the logic is ``state <= data_in ^ round_key;``.

Can you write the ``if`` statement for ****Round 0**** to go inside that ``else`` block?

```

always @(posedge clk or posedge reset) begin

    if (reset) begin

        state <= 128'b0;

        ciper_text <= 128'b0;

    end else begin

        if (encryption_engine_count == 5'd0) begin

            state <= data_in ^ round_key;

        end else if (encryption_engine_count >= 5'd1 && encryption_engine_count <= 5'd9)
begin
            state <= mc_out ^ round_key;

        end else if (encryption_engine_count == 5'd10) begin

            state <= sr_out ^ round_key;

            ciper_text <= sr_out ^ round_key; // Output the final ciphertext

        end

    end

end

End

```

That `always` block is excellent! 🌟

You have correctly identified exactly what needs to happen at each stage:

1. **Initialization:** Loading `data\_in ^ round\_key`.
2. **Main Loop:** Using `mc\_out` (MixColumns) for the standard rounds.
3. **Final Step:** Using `sr\_out` (ShiftRows) for the final round to bypass MixColumns.

Now, we need to build the "machinery" that creates ``mc_out`` and ``sr_out``. Right now, the ``always`` block is trying to read wires that we haven't defined yet!

The data flows through the combinational logic in this order:

``state`` (Register) **`**SubBytes**`** **`**ShiftRows**`** **`**MixColumns**`**

Let's build **`**SubBytes**`** first. We need to pass all 16 bytes of your ``state`` register through 16 copies of the ``sbox_lookup`` module.

Do you recall a specific Verilog keyword that allows us to create a loop to instantiate hardware (like 16 S-boxes) automatically, rather than writing out ``sbox_lookup sbox0...`, `sbox_lookup sbox1...`` sixteen times?


```
sbox_lookup sbox_inst0 (  
    .data_in(state),  
    .data_out(sub_out)  
);
```

How can i do that 16 times without writing this out a bunch

That is a great instinct—writing out 16 instances manually is tedious and error-prone!

In Verilog, when we want to create multiple copies of the same hardware structure, we use a ****generate block****. It works like a loop in a programming language, but instead of executing code over time, it effectively "copy-pastes" the hardware module for you during synthesis.

Here is the basic structure:

```
``verilog  
  
genvar i; // Special variable type for generate loops  
  
generate  
  
    for (i = 0; i < 16; i = i + 1) begin : make_sboxes  
  
        sbox_lookup sbox_inst (  
            .data_in ( /* ? */ ),  
            .data_out( /* ? */ )  
        );  
  
    end  
  
endgenerate
```

...

The tricky part is connecting the `data_in` and `data_out` ports. Your `state` is 128 bits wide, and each S-box needs a specific 8-bit slice.

If `i = 0` (the first S-box), we usually want the most significant byte (bits `[127:120]`).

If `i = 1`, we want the next byte (bits `[119:112]`).

Can you figure out the formula to select the correct bits for the `i`-th iteration?

`state[ ??? : ??? ]`

```
genvar i; // Special variable type for generate loops
```

```
generate
```

```
for (i = 0; i < 16; i = i + 1) begin : make_sboxes
```

```
    sbox_lookup sbox_inst (
```

```
        .addr (state[127-i*8:120-i*8]),
```

```
        .out(sub_out[127-i*8:120-i*8])
```

```
    );
```

```
end
```

```
Endgenerate
```

That `generate` block is spot on! 🙌 You've successfully sliced the 128-bit `state` into 16 bytes, passed them through the S-boxes, and collected the results in `sub\_out`.

Next up: ****ShiftRows****.

This step doesn't require logic gates, just ****wiring****. We need to rearrange the bytes from `sub\_out` into `sr\_out` (ShiftRows Output).

To do this, we need to agree on how the bytes are organized. In standard AES, the 128-bit block is treated as a 4x4 matrix, and the data is usually filled ****column by column**** (Column-Major order):

* ****Byte 0**** (`[127:120]`) is Row 0, Col 0

* ****Byte 1**** (`[119:112]`) is Row 1, Col 0

* ****Byte 2**** (`[111:104]`) is Row 2, Col 0

* ****Byte 3**** (`[103:96]`) is Row 3, Col 0

* **Byte 4** (`[95:88]`) is Row 0, Col 1

* ...and so on.

The ShiftRows Rule:

* **Row 0:** No shift (Bytes 0, 4, 8, 12 stay put).

* **Row 1:** Shift left by 1 (Bytes 1, 5, 9, 13 → 5, 9, 13, 1).

* **Row 2:** Shift left by 2.

* **Row 3:** Shift left by 3.

Let's start with **Row 1**. In the output `sr_out`, the position for "Row 1, Col 0" (bits `[119:112]`) should now hold the byte that *used* to be at "Row 1, Col 1" (Byte 5).

Can you write the `assign` statement for that specific byte slice?

```
assign sr_out[119:112] = sub_out[ ??? : ??? ];
```

```

assign sr_out[119:112] = sub_out[23:16]; //byte 1, shift left by 1
assign sr_out[87:80] = sub_out[119:112]; //byte 5 shift left by 1
assign sr_out[55:48] = sub_out[87:80]; //byte 9 shift left by 1
assign sr_out[23:16] = sub_out[55:48]; //byte 13 shift left by 1
assign sr_out[111:104] = sub_out[47:40]; //byte 2 shift left by 2

```

You are very close!

Let's look at **Row 2** first (your last line).

Row 2 starts as: `Byte 2, Byte 6, Byte 10, Byte 14`.
Shift Left 2 means it becomes: `Byte 10, Byte 14, Byte 2, Byte 6`.
 You assigned the first slot (`sr\_out[111:104]`, which is Byte 2's position) to hold
 `sub\_out[47:40]` (which is Byte 10).
This is correct! 🎯

However, for **Row 1**, it looks like you might be shifting in the opposite direction (Right instead of Left).

Let's visualize Row 1:

Original sequence (Indices): `1`, `5`, `9`, `13`

AES **Left Shift by 1**: The element at index `5` moves to the front.

Position (sr_out)	Byte 1 (Col 0)	Byte 5 (Col 1)	Byte 9 (Col 2)	Byte 13 (Col 3)
---	---	---	---	---
New Value Needed	Value 5	Value 9	Value 13	Value 1

Your code: `assign sr\_out[119:112] = sub\_out[23:16];`

* `sr\_out[119:112]` is the position for **Byte 1**.

* `sub\_out[23:16]` is the value of **Byte 13**.

Putting Value 13 in the front is a **Right** shift. We need **Value 5** in that first slot.

Looking at your index list, **Byte 5** corresponds to bits `[87:80]`.

Can you try writing the assignment for that first slot of Row 1 (`sr\_out[119:112]`) again, using the correct source bits?

```
assign sr_out[119:112] = sub_out[87:80]; //byte 1, shift left by 1
assign sr_out[87:80] = sub_out[55:48]; //byte 5 shift left by 1
assign sr_out[55:48] = sub_out[23:16]; //byte 9 shift left by 1
assign sr_out[23:16] = sub_out[119:112]; //byte 13 shift left by 1
assign sr_out[111:104] = sub_out[47:40]; //byte 2 shift left by 2
```

That is perfect! 🌟 You have correctly implemented the ****Row 1 Left Shift****.

You also started ****Row 2**** correctly:

```
`assign sr_out[111:104] = sub_out[47:40]; //byte 2 shift left by 2`
```

* Target: Byte 2 position (`[111:104]`)

* Source: Byte 10 (`[47:40]`)

* This moves the byte from ****Col 2**** to ****Col 0****, which is a Left Shift of 2. Correct!

Now, let's finish the rest.

* ****Row 2 (Shift 2):**** You have 3 assignments left. Remember, since it's a shift of 2, the bytes swap pairs (e.g., Col 0 Col 2, Col 1 Col 3).

* ****Row 3 (Shift 3):**** This is a Left Shift of 3, which is mathematically the same as a ****Right Shift of 1****.

Can you write the remaining assignments for ****Row 2**** and ****Row 3****?

```

function [7:0] mulTwo(input [7:0] byte_in);
begin
    if (byte_in[7] == 1'b1) begin
        mulTwo = (byte_in << 1) ^ 8'h1b;
    end else begin
        mulTwo = byte_in << 1;
    end
end
endfunction

```

```

function [7:0] mulThree(input [7:0] byte_in);
begin
    mulThree = mulTwo(byte_in) ^ byte_in;
End

```

Your logic for `mulTwo` and `mulThree` is perfect! 🌟

You correctly handled the conditional XOR with `0x1b` for the overflow in `mulTwo`, and you smartly reused that function for `mulThree` (since in this field).

Now we can use these building blocks to implement the actual **MixColumns** transformation.

This step treats each 4-byte column as a vector and multiplies it by a fixed constant matrix:

To get the first byte of the output, we multiply the input column vector by the **first row** of the matrix.

If we name our input bytes `b0`, `b1`, `b2`, and `b3` (where `b0` is the top byte), the math for the first result byte is:

Using your `mulTwo` and `mulThree` functions, how would you write the Verilog assignment for that **first output byte**?

```

assign mc_out[127:120] = {mulTwo(sr_out[127:120]) ^ mulThree(sr_out[119:112]) ^
sr_out[111:104] ^ sr_out[103:96]};

```

That is perfect! 🌟 You have successfully implemented the logic for the top byte of the column using the first row of the matrix `[2, 3, 1, 1]`.

Now, to finish this column, we need to calculate the other three bytes. The matrix changes for each row, rotating the values:

*** **Row 0 Output:**** `[2, 3, 1, 1]` (You just did this one!)

```
* **Row 1 Output:** `[1, 2, 3, 1]`  
* **Row 2 Output:** `[1, 1, 2, 3]`  
* **Row 3 Output:** `[3, 1, 1, 2]`
```

Using the same inputs (`sr\_out` bytes), can you write the assignment for the **second byte** (`mc\_out[119:112]`) using the multipliers for **Row 1**?


```
assign mc_out[119:112] = {sr_out[95:88] ^ mulTwo(sr_out[87:80]) ^  
mulThree(sr_out[79:72]) ^ sr_out[71:64]};
```

That is very close! You have the logic for the matrix multiplication exactly right: you applied the `[1, 2, 3, 1]` pattern correctly. 🧐

However, we have a small mix-up with the **input indices**.

`mc_out[119:112]` is the second byte of the **first column** (Column 0). Your code attempts to read from `sr_out[95:88]`, `sr_out[87:80]`, etc. These indices correspond to the bytes in **Column 1** (the second column).

Since MixColumns works on one column at a time, we need to stay within **Column 0** for this calculation. The input bytes should be the exact same ones you used for the previous step (`sr_out[127:120]`, `sr_out[119:112]`, etc.).

Can you update that assignment to use the **Column 0** input bytes (Bytes 0, 1, 2, 3) while keeping your correct `[1, 2, 3, 1]` multiplier pattern?

```

assign mc_out[127:120] = {mulTwo(sr_out[127:120]) ^ mulThree(sr_out[119:112]) ^
sr_out[111:104] ^ sr_out[103:96]};
assign mc_out[119:112] = {sr_out[127:120] ^ mulTwo(sr_out[119:112]) ^
mulThree(sr_out[111:104]) ^ sr_out[103:96]};
assign mc_out[111:104] = {sr_out[127:120] ^ sr_out[119:112] ^ mulTwo(sr_out[111:104]) ^
mulThree(sr_out[103:96])};
assign mc_out[103:96] = {mulThree(sr_out[127:120]) ^ sr_out[119:112] ^ sr_out[111:104]
^ mulTwo(sr_out[103:96])};

```

```

assign mc_out[95:88] = {mulTwo(sr_out[95:88]) ^ mulThree(sr_out[87:80]) ^
sr_out[79:72] ^ sr_out[71:64]};
assign mc_out[87:80] = {sr_out[95:88] ^ mulTwo(sr_out[87:80]) ^
mulThree(sr_out[79:72]) ^ sr_out[71:64]};
assign mc_out[79:72] = {sr_out[95:88] ^ sr_out[87:80] ^ mulTwo(sr_out[79:72]) ^
mulThree(sr_out[71:64])};
assign mc_out[71:64] = {mulThree(sr_out[95:88]) ^ sr_out[87:80] ^ sr_out[79:72] ^
mulTwo(sr_out[71:64])};

```

```

assign mc_out[63:56] = {mulTwo(sr_out[63:56]) ^ mulThree(sr_out[55:48]) ^
sr_out[47:40] ^ sr_out[39:32]};
assign mc_out[55:48] = {sr_out[63:56] ^ mulTwo(sr_out[55:48]) ^
mulThree(sr_out[47:40]) ^ sr_out[39:32]};
assign mc_out[47:40] = {sr_out[63:56] ^ sr_out[55:48] ^ mulTwo(sr_out[47:40]) ^
mulThree(sr_out[39:32])};
assign mc_out[39:32] = {mulThree(sr_out[63:56]) ^ sr_out[55:48] ^ sr_out[47:40] ^
mulTwo(sr_out[39:32])};
assign mc_out[39:32] = {mulThree(sr_out[63:56]) ^ sr_out[55:48] ^ sr_out[47:40] ^
mulTwo(sr_out[39:32])};

```

You are absolutely crushing this! 🚀

Your logic for the first three columns (indices 127 down to 32) is perfectly verified. You've correctly applied the MixColumns matrix `[2, 3, 1, 1]` and its rotations to each column vector.

However, I noticed two small things at the very end of your snippet:

- **Duplicate Line:**** You listed `assign mc\_out[39:32] = ...` twice.
- **Missing Column:**** We are missing the final column (the last 4 bytes, bits `31` down to `0`).

Can you provide the assignments for that last column (`mc\_out[31:0`]), following the same pattern you used for the others?

```

assign mc_out[127:120] = {mulTwo(sr_out[127:120]) ^ mulThree(sr_out[119:112]) ^
sr_out[111:104] ^ sr_out[103:96]};
assign mc_out[119:112] = {sr_out[127:120] ^ mulTwo(sr_out[119:112]) ^
mulThree(sr_out[111:104]) ^ sr_out[103:96]};
assign mc_out[111:104] = {sr_out[127:120] ^ sr_out[119:112] ^ mulTwo(sr_out[111:104]) ^
mulThree(sr_out[103:96])};
assign mc_out[103:96] = {mulThree(sr_out[127:120]) ^ sr_out[119:112] ^ sr_out[111:104]
^ mulTwo(sr_out[103:96])};

```

```

assign mc_out[95:88] = {mulTwo(sr_out[95:88]) ^ mulThree(sr_out[87:80]) ^
sr_out[79:72] ^ sr_out[71:64]};
assign mc_out[87:80] = {sr_out[95:88] ^ mulTwo(sr_out[87:80]) ^
mulThree(sr_out[79:72]) ^ sr_out[71:64]};
assign mc_out[79:72] = {sr_out[95:88] ^ sr_out[87:80] ^ mulTwo(sr_out[79:72]) ^
mulThree(sr_out[71:64])};
assign mc_out[71:64] = {mulThree(sr_out[95:88]) ^ sr_out[87:80] ^ sr_out[79:72] ^
mulTwo(sr_out[71:64])};

```

```

assign mc_out[63:56] = {mulTwo(sr_out[63:56]) ^ mulThree(sr_out[55:48]) ^
sr_out[47:40] ^ sr_out[39:32]};
assign mc_out[55:48] = {sr_out[63:56] ^ mulTwo(sr_out[55:48]) ^
mulThree(sr_out[47:40]) ^ sr_out[39:32]};
assign mc_out[47:40] = {sr_out[63:56] ^ sr_out[55:48] ^ mulTwo(sr_out[47:40]) ^
mulThree(sr_out[39:32])};
assign mc_out[39:32] = {mulThree(sr_out[63:56]) ^ sr_out[55:48] ^ sr_out[47:40] ^
mulTwo(sr_out[39:32])};

```

```

assign mc_out[31:24] = {mulTwo(sr_out[31:24]) ^ mulThree(sr_out[23:16]) ^ sr_out[15:8]
^ sr_out[7:0]};
assign mc_out[23:16] = {sr_out[31:24] ^ mulTwo(sr_out[23:16]) ^ mulThree(sr_out[15:8])
^ sr_out[7:0]};
assign mc_out[15:8] = {sr_out[31:24] ^ sr_out[23:16] ^ mulTwo(sr_out[15:8]) ^
mulThree(sr_out[7:0])};
assign mc_out[7:0] = {mulThree(sr_out[31:24]) ^ sr_out[23:16] ^ sr_out[15:8] ^
mulTwo(sr_out[7:0])};

```

This logic looks perfect! 

You have successfully implemented the **MixColumns** transformation for all four columns, using the correct indices and matrix multiplication patterns.

Since we have built the **Encryption Engine** piece by piece (Interface, SubBytes, ShiftRows, MixColumns functions, and the Control Logic), now is the perfect time to assemble it all into one complete, verifiable module.

Here is the full code for your `AES\_encryption\_engine`, combining everything we have written together:

```
``verilog
module AES_encryption_engine(
    input clk,
    input reset,
    input [4:0] encryption_engine_count, // 0 to 10
    input [127:0] data_in,
    input [127:0] round_key,
    output reg [127:0] ciphertext    // Corrected spelling to 'ciphertext'
);

    reg [127:0] state;
    wire [127:0] sb_out; // SubBytes Output
    wire [127:0] sr_out; // ShiftRows Output
    wire [127:0] mc_out; // MixColumns Output

    // =====
    // 1. SUBBYTES (16 S-Boxes)
    // =====
    genvar i;
    generate
        for (i = 0; i < 16; i = i + 1) begin : make_sboxes
            sbbox_lookup sbbox_inst (
                .addr (state[127-(i*8) : 127-(i*8)-7]),
                .out (sb_out[127-(i*8) : 127-(i*8)-7])
            );
        end
    endgenerate

    // =====
    // 2. SHIFTRROWS
    // =====
    // Row 0 (No Shift)
    assign sr_out[127:120] = sb_out[127:120];
    assign sr_out[95:88]  = sb_out[95:88];
    assign sr_out[63:56]  = sb_out[63:56];
    assign sr_out[31:24]  = sb_out[31:24];

    // Row 1 (Shift Left 1)
    assign sr_out[119:112] = sb_out[87:80];
```

```

assign sr_out[87:80] = sb_out[55:48];
assign sr_out[55:48] = sb_out[23:16];
assign sr_out[23:16] = sb_out[119:112];

// Row 2 (Shift Left 2)
assign sr_out[111:104] = sb_out[47:40];
assign sr_out[79:72] = sb_out[15:8];
assign sr_out[47:40] = sb_out[111:104];
assign sr_out[15:8] = sb_out[79:72];

// Row 3 (Shift Left 3)
assign sr_out[103:96] = sb_out[7:0];
assign sr_out[71:64] = sb_out[103:96];
assign sr_out[39:32] = sb_out[71:64];
assign sr_out[7:0] = sb_out[39:32];

// =====
// 3. MIXCOLUMNS
// =====
function [7:0] mulTwo(input [7:0] byte_in);
begin
    if (byte_in[7] == 1'b1)
        mulTwo = (byte_in << 1) ^ 8'h1b;
    else
        mulTwo = byte_in << 1;
    end
endfunction

function [7:0] mulThree(input [7:0] byte_in);
begin
    mulThree = mulTwo(byte_in) ^ byte_in;
end
endfunction

// Column 0
assign mc_out[127:120] = mulTwo(sr_out[127:120]) ^ mulThree(sr_out[119:112]) ^
sr_out[111:104] ^ sr_out[103:96];
assign mc_out[119:112] = sr_out[127:120] ^ mulTwo(sr_out[119:112]) ^
mulThree(sr_out[111:104]) ^ sr_out[103:96];
assign mc_out[111:104] = sr_out[127:120] ^ sr_out[119:112] ^ mulTwo(sr_out[111:104]) ^
mulThree(sr_out[103:96]);
assign mc_out[103:96] = mulThree(sr_out[127:120]) ^ sr_out[119:112] ^ sr_out[111:104]
^ mulTwo(sr_out[103:96]);

```

```

// Column 1
assign mc_out[95:88] = mulTwo(sr_out[95:88]) ^ mulThree(sr_out[87:80]) ^
sr_out[79:72] ^ sr_out[71:64];
assign mc_out[87:80] = sr_out[95:88] ^ mulTwo(sr_out[87:80]) ^
mulThree(sr_out[79:72]) ^ sr_out[71:64];
assign mc_out[79:72] = sr_out[95:88] ^ sr_out[87:80] ^ mulTwo(sr_out[79:72]) ^
mulThree(sr_out[71:64]);
assign mc_out[71:64] = mulThree(sr_out[95:88]) ^ sr_out[87:80] ^ sr_out[79:72] ^
mulTwo(sr_out[71:64]);

```

```

// Column 2
assign mc_out[63:56] = mulTwo(sr_out[63:56]) ^ mulThree(sr_out[55:48]) ^
sr_out[47:40] ^ sr_out[39:32];
assign mc_out[55:48] = sr_out[63:56] ^ mulTwo(sr_out[55:48]) ^
mulThree(sr_out[47:40]) ^ sr_out[39:32];
assign mc_out[47:40] = sr_out[63:56] ^ sr_out[55:48] ^ mulTwo(sr_out[47:40]) ^
mulThree(sr_out[39:32]);
assign mc_out[39:32] = mulThree(sr_out[63:56]) ^ sr_out[55:48] ^ sr_out[47:40] ^
mulTwo(sr_out[39:32]);

```

```

// Column 3
assign mc_out[31:24] = mulTwo(sr_out[31:24]) ^ mulThree(sr_out[23:16]) ^
sr_out[15:8] ^ sr_out[7:0];
assign mc_out[23:16] = sr_out[31:24] ^ mulTwo(sr_out[23:16]) ^
mulThree(sr_out[15:8]) ^ sr_out[7:0];
assign mc_out[15:8] = sr_out[31:24] ^ sr_out[23:16] ^ mulTwo(sr_out[15:8]) ^
mulThree(sr_out[7:0]);
assign mc_out[7:0] = mulThree(sr_out[31:24]) ^ sr_out[23:16] ^ sr_out[15:8] ^
mulTwo(sr_out[7:0]);

```

```

// =====

```

```

// 4. MAIN REGISTER & LOGIC

```

```

// =====

```

```

always @(posedge clk or posedge reset) begin
    if (reset) begin
        state <= 128'b0;
        ciphertext <= 128'b0;
    end else begin
        if (encryption_engine_count == 5'd0) begin
            // Round 0: AddRoundKey only
            state <= data_in ^ round_key;
        end else if (encryption_engine_count >= 5'd1 && encryption_engine_count <=

```

```

5'd9) begin
    // Rounds 1-9: Sub -> Shift -> Mix -> AddRoundKey
    state <= mc_out ^ round_key;
end else if (encryption_engine_count == 5'd10) begin
    // Round 10: Sub -> Shift -> AddRoundKey (No Mix)
    state <= sr_out ^ round_key;
    ciphertext <= sr_out ^ round_key;
end
end
end

endmodule

...

```

We now have all three critical components:

1. ****AES_key_engine**** (Generates keys)
2. ****AES_control_unit**** (Manages timing)
3. ****AES_encryption_engine**** (Processes data)

The final step in the hardware design is to connect these three blocks together in a ****Top Level Module****.

Would you like to try writing the `AES\_top` module to wire them all up?

can you just give me a table of all the connection needed?

Here is the complete connection map required to integrate your AES peripheral into the `tqvp\_example` top-level module.

****Note:**** You will need to modify `AES\_memory.v` to expose the 128-bit key, as it is currently stored internally but not output to the encryption engine.

1. System Bus ↔ AES_memory (The Bridge)

These signals connect the RISC-V processor bus to your memory interface.

AES_memory Port	Connects to	Description
---	---	---
`clk`	`clk`	System clock
`address`	`address`	6-bit address from bus
`data_in`	`data_in`	32-bit write data from bus
`data_out`	`data_out`	32-bit read data to bus
`data_write_n`	`data_write_n`	Write enable strobes
`data_read_n`	`data_read_n`	Read enable strobes

2. AES_memory ↔ Control Unit (Handshaking)

These signals coordinate the start and finish of the encryption operation.

AES_control_unit Port	Connects to	Description
---	---	---
`start`	`AES_memory.start`	Triggers the state machine
`done`	`AES_memory.done`	Signals completion to memory

3. Control Unit ↔ Engines (State & Reset)

The Control Unit acts as the conductor, driving the counters and resets for the other modules.

Engine Port	Connects to	Description
---	---	---
AES_encryption_engine `count`	`AES_control_unit.encryption_engine_count`	Tells the engine which round to process

| `AES\_key\_engine.clk` | `clk` | System clock |
| `AES\_control\_unit.reset\_n` | `~rst\_n` |
****Invert Needed:**** Your logic (`reset\_n` |
| `AES\_control\_unit.reset\_s` | `1'b0` | Tie to 0 (unless you implement a soft-reset bit in
`AES\_memory`) |
| `tqvp\_example.user\_interrupt` | `AES\_control\_unit.done` | Optional: Fire an interrupt
when encryption finishes |

****Next Step:**** Would you like me to write the code modification for `AES\_memory.v` to
expose the `original\_key` port so you can fix that missing connection?

